Rethinking Link Prediction for Directed Graphs

Mingguo He, *Member, IEEE*, Yuhe Guo, Yanping Zheng, Zhewei Wei, *Member, IEEE*, Stephan Günnemann, and Xiaokui Xiao, *Fellow, IEEE*

***Abstract*—Link prediction for directed graphs is a crucial task with diverse real-world applications. Recent advances in embedding methods and Graph Neural Networks (GNNs) have shown promising improvements. However, these methods often lack a thorough analysis of their expressiveness and suffer from the lack of effective benchmarks for fair evaluation. In this paper, we propose a unified framework to assess the expressiveness of existing methods, highlighting the impact of dual embeddings and decoder design on directed link prediction performance. To address limitations in current benchmark setups, we introduce DirLinkBench, a robust new benchmark with comprehensive coverage, standardized evaluation, and modular extensibility. The results on DirLinkBench show that current methods struggle to achieve strong performance, while DiGAE outperforms other baselines overall. We further revisit DiGAE theoretically, showing its graph convolution aligns with GCN on an undirected bipartite graph. Inspired by these insights, we propose a novel Spectral Directed Graph Auto-Encoder SDGAE that achieves state-of-the-art average performance on DirLinkBench. Finally, we analyze key factors influencing directed link prediction and highlight open challenges in this field.**

***Index Terms*—Directed graph, link prediction, benchmark, spectral-based graph neural network.**

I. Introduction

A DIRECTED graph (or digraph) is a type of graph in which the edges between nodes have a specific direction. These graphs are commonly used to model real-world asymmetric relationships, such as "following" and "followed" in social networks [1] or "link" and "linked" in web pages [2]. Directed graphs capture the inherent directionality of these relationships and provide a more accurate representation of complex systems. Link prediction is a fundamental and widely studied task in directed graphs, with numerous real-world applications. Examples include predicting follower relationships in social networks [3], recommending products in e-commerce [4], and detecting intrusions in network security [5].

Machine learning techniques have been extensively developed to enhance link prediction performance on directed graphs. Existing methods can be broadly categorized into embedding methods and graph neural networks (GNNs). Embedding methods aim to preserve the asymmetry of directed graphs by generating two separate embeddings for each node u: a source embedding $\mathbf{s}_u$ and a target embedding $\mathbf{t}_u$ [12], [13], which are also known as content/context representations [6], [9], [27]. GNNs, on the other hand, can be further divided into four classes based on the types of embeddings they generate. (1) Source-target methods, similar to embedding methods, employ specialized propagation mechanisms to learn distinct source and target embeddings for each node [14], [15]. (2) Gravity-inspired methods, inspired by Newton's law of universal gravitation, learn a real-valued embedding $\mathbf{h}_u \in \mathbb{R}^d$ and a mass parameter $m_u \in \mathbb{R}^+$ for each node u [17], [18]. (3) Single real-valued methods follow a conventional approach by learning a single real-valued embedding $\mathbf{h}_u \in \mathbb{R}^d$ [21], [23]. (4) Complex-valued methods use Hermitian adjacency matrices, learning complex-valued embeddings $\mathbf{z}_u \in \mathbb{C}^d$ [24], [26]. We summarize these methods in Table I.

Although the methods described above have achieved promising results in directed link prediction, several challenges remain. First, it is unclear what types of embedding are effective in predicting directed links, as there is a lack of comprehensive research assessing these methods' expressiveness. Second, existing methods have not been fairly compared and evaluated, highlighting the need for a robust benchmark for directed link prediction. Current experimental setups face multiple issues, such as the omission of basic baselines (e.g., MLP shown in Figs. 1(a) and 1(b)), label leakage (illustrated in Tables V and VI), class imbalance (shown in Figs. 2(a) and 2(b)), single evaluation metrics, and inconsistent dataset splits. We discuss these issues in detail in Section III-B.

In this paper, we first propose a unified learning framework for directed link prediction methods to assess the expressiveness of different embedding types. We demonstrate that dual methods (including source-target, gravity-inspired, and complex-valued methods) are more expressive for directed link prediction than

Received 17 May 2025; revised 9 March 2026; accepted 25 April 2026. Date of publication 29 April 2026; date of current version 6 August 2026. This work was supported in part by the National Natural Science Foundation of China under Grant U2241212 and Grant 92470128, in part by the Ministry of Education, Singapore, under an AcRF Tier 2 under Grant MOE-000761-01, in part by the fund for Building World-Class Universities (Disciplines) of Renmin University of China, in part by the Engineering Research Center of Next-Generation Intelligent Search and Recommendation, Ministry of Education, in part by Intelligent Social Governance Interdisciplinary Platform, in part by the Major Innovation & Planning Interdisciplinary Platform for the "Double-First Class" Initiative, in part by Public Policy and Decision-Making Research Lab, and in part by Public Computing Cloud, Renmin University of China. Recommended for acceptance by S. J. Pan. *(Corresponding author: Zhewei Wei.)*

Mingguo He was with the Gaoling School of Artificial Intelligence, Renmin University of China, Beijing 100872, China. He is now with the School of Computing, National University of Singapore, Singapore 117417 (e-mail: mingguo@nus.edu.sg).

Yuhe Guo, Yanping Zheng, and Zhewei Wei are with the Gaoling School of Artificial Intelligence, Renmin University of China, Beijing 100872, China (e-mail: guoyuhe@ruc.edu.cn; zhengyanping@ruc.edu.cn; zhewei@ruc.edu.cn).

Stephan Günnemann is with the Technical University of Munich, 80333 Munich, Germany (e-mail: s.guennemann@tum.de).

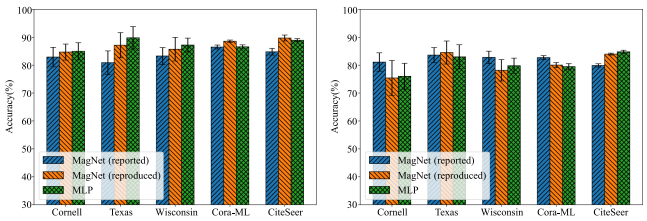
Xiaokui Xiao is with the School of Computing, National University of Singapore, Singapore 117417 (e-mail: xkxiao@nus.edu.sg).

Our code is available at https://github.com/ivam-he/DirLinkBench-SDGAE.

Digital Object Identifier 10.1109/TPAMI.2026.3688944

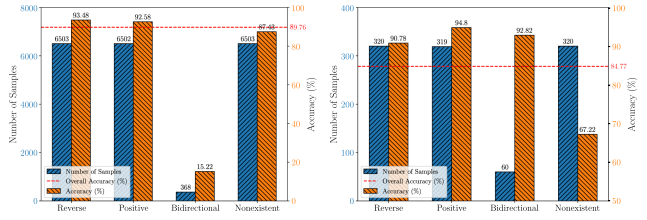
TABLE I
OVERVIEW OF DIRECTED GRAPH LEARNING METHODS

Method	Name	Embedding
Embedding Methods	HOPE [6]	$\mathbf{s}_u, \mathbf{t}_u$
	APP [7]	$\mathbf{s}_u, \mathbf{t}_u$
	AROPE [8]	$\mathbf{s}_u, \mathbf{t}_u$
	STRAP [9]	$\mathbf{s}_u, \mathbf{t}_u$
	NERD [10]	$\mathbf{s}_u, \mathbf{t}_u$
	DGGAN [11]	$\mathbf{s}_u, \mathbf{t}_u$
	ELTRA [12]	$\mathbf{s}_u, \mathbf{t}_u$
	ODIN [13]	$\mathbf{s}_u, \mathbf{t}_u$
Graph Neural Networks	DiGAE [14]	$\mathbf{s}_u, \mathbf{t}_u$
	CoBA [15]	$\mathbf{s}_u, \mathbf{t}_u$
	BLADE [16]	$\mathbf{s}_u, \mathbf{t}_u$
	Gravity GAE [17]	$\mathbf{h}_u, m_u$
	DHYPR [18]	$\mathbf{h}_u, m_u$
	DGCN [19]	$\mathbf{h}_u$
	DiGCN & DiGCNIB [20]	$\mathbf{h}_u$
	DirGNN [21]	$\mathbf{h}_u$
	HoloNets [22]	$\mathbf{h}_u$
	NDDGNN [23]	$\mathbf{h}_u$
	MagNet [24]	$\mathbf{z}_u$
	LightDiC [25]	$\mathbf{z}_u$
	DUPLEX [26]	$\mathbf{z}_u$



(a) Direction Prediction (b) Existence Prediction

Fig. 1. The results of MagNet [24] as reported in the original paper, alongside the reproduced MagNet and MLP results.



(a) Cora (b) CiteSeer

Fig. 2. The number of samples and the accuracy for each class of DUPLEX [26] on the Cora and CiteSeer dataset in the 4 C task.

single methods (i.e., single real-valued methods). Meanwhile, we highlight the often-overlooked importance of decoder design in achieving better performance, as most research has primarily focused on encoders. To address the limitations of existing experimental setups, we introduce DirLinkBench, a robust new benchmark for directed link prediction that offers comprehensive coverage (seven real-world datasets, 16 baseline methods, and seven evaluation metrics), standardized evaluation (unified splits, feature inputs, and task setups), and modular extensibility (support for adding new datasets, decoders, and sampling strategies).

The results in DirLinkBench reveal that current methods struggle to achieve strong and consistent performance across diverse datasets. Interestingly, a simple directed graph auto-encoder, DiGAE [14], outperforms other baselines in general. We revisit DiGAE from a theoretical perspective and observe that its graph convolution is equivalent to the GCN [28] convolution on an undirected bipartite graph. Building on this insight, we propose SDGAE, a novel spectral directed graph auto-encoder that learns arbitrary spectral filters via polynomial approximation. SDGAE achieves state-of-the-art (SOTA) results on four of the seven datasets and ranks highest on average. Finally, we investigate key factors influencing directed link prediction (e.g., input features, decoder design, and degree distribution) and conclude with open challenges to advance the field. We summarize our contributions as follows.

- We propose a unified framework to assess the expressiveness of directed link prediction methods, showing that dual methods are more expressive and highlighting the importance of decoder design.
- We introduce DirLinkBench, a robust new benchmark with comprehensive coverage, standardized evaluation, and modular extensibility for directed link prediction.
- We propose a novel directed graph auto-encoder SDGAE, inspired by the theoretical insights of DiGAE and achieving SOTA average performance on DirLinkBench.
- We empirically analyze the factors affecting the performance of directed link prediction and highlight open challenges for future research.

II. BACKGROUND AND RELATED WORK

In this section, we first introduce the background and related work of directed graph learning methods, which can be broadly categorized into embedding methods and graph neural networks (GNNs). Additionally, we will introduce some background on spectral-based GNNs for undirected graphs.

Notation: We consider a directed, unweighted graph $\mathcal{G} = (V, E)$, with node set V and edge set E. Let $n = |V|$ and $m = |E|$ represent the number of nodes and edges in $\mathcal{G}$, respectively. We use $\mathbf{A}$ to denote the adjacency matrix of $\mathcal{G}$, where $\mathbf{A}_{uv} = 1$ if there exists a directed edge from node u to node v, and $\mathbf{A}_{uv} = 0$ otherwise. The Hermitian adjacency matrix of $\mathcal{G}$ is denoted by $\mathbf{H}$ and defined as $\mathbf{H} = \mathbf{A}_\mathrm{s} \odot \exp(i\frac{\pi}{2}\mathbf{\Theta})$. Here, $\mathbf{A}_\mathrm{s} = \mathbf{A} \cup \mathbf{A}^\top$ is the adjacency matrix of the undirected graph derived from $\mathcal{G}$, and $\mathbf{\Theta} = \mathbf{A} - \mathbf{A}^\top$ is a skew-symmetric matrix. i is the imaginary unit. We denote the out-degree and in-degree matrices of $\mathbf{A}$ by $\mathbf{D}_\mathrm{out} = \mathrm{diag}(\mathbf{A}\mathbf{1})$ and $\mathbf{D}_\mathrm{int} = \mathrm{diag}(\mathbf{A}^\top\mathbf{1})$, respectively, where $\mathbf{1}$ is the all-one vector. Let $\mathbf{X} \in \mathbb{R}^{n\times d'}$ denote the node feature matrix, where each node has a d'-dimensional feature vector.

A. Embedding Methods

Embedding methods for directed graphs aim to capture asymmetric relationships. Most approaches assign each node

TABLE II
A UNIFIED FRAMEWORK FOR DIRECTED LINK PREDICTION METHODS

Encoder $\mathrm{Enc}(\cdot)$	Embeddings $(\boldsymbol{\theta}_u, \boldsymbol{\phi}_u)$	Possible Decoder $\mathrm{Dec}(\cdot)$		
Single real-valued	$\mathbf{h}_u = \boldsymbol{\theta}_u, \quad \varnothing = \boldsymbol{\phi}_u$	$\sigma(\mathbf{h}_u^\top \mathbf{h}_v)$;	$\mathrm{MLP}(\mathbf{h}_u \odot \mathbf{h}_v)$;	$\mathrm{MLP}(\mathbf{h}_u \Vert \mathbf{h}_v)$
Source-target	$\mathbf{s}_u = \boldsymbol{\theta}_u, \quad \mathbf{t}_u = \boldsymbol{\phi}_u$	$\sigma(\mathbf{s}_u^\top \mathbf{t}_v)$;	$\mathrm{MLP}(\mathbf{s}_u \odot \mathbf{t}_v)$;	$\mathrm{LR}(\mathbf{s}_u \Vert \mathbf{t}_v)$
Complex-valued	$\mathbf{z}_u = \boldsymbol{\theta}_u \odot \exp(i\,\boldsymbol{\phi}_u)$	$\mathrm{Direc}(\mathbf{z}_u, \mathbf{z}_v)$;	$\mathrm{MLP}(\boldsymbol{\theta}_u \Vert \boldsymbol{\theta}_v \Vert \boldsymbol{\phi}_u \Vert \boldsymbol{\phi}_v)$	
Gravity-inspired	$\mathbf{h}_u = \boldsymbol{\theta}_u, \quad m_u = g(\boldsymbol{\phi}_u)$	$\sigma(m_v - \lambda \log \Vert \mathbf{h}_u - \mathbf{h}_v \Vert_2^2)$;	$\sigma(m_v - \lambda \log(\mathrm{dist}(\mathbf{h}_u, \mathbf{h}_v)))$	

TABLE III
THE RESULTS OF MLP AND BASELINES UNDER PYGSD [42] SETUP ON DIRECTION PREDICTION (DP) TASK

Method	Cora-ML	CiteSeer	Telegram	Cornell	Texas	Wisconsin
MLP	86.13±0.45	85.51±0.63	95.61±0.15	**86.40±2.60**	**83.08±4.33**	**87.95±2.65**
DGCN	85.49±0.75	84.85±0.56	96.03±0.35	84.55±3.71	79.59±5.09	84.57±3.59
DiGCN	85.37±0.54	83.88±0.82	94.95±0.54	85.41±2.78	76.57±3.98	82.41±2.56
DiGCNIB	86.12±0.42	85.58±0.56	95.99±0.44	86.28±3.37	82.27±3.51	87.07±2.16
MagNet	**86.33±0.54**	**85.80±0.63**	**96.97±0.21**	83.29±4.28	80.25±4.78	86.60±2.72

TABLE IV
THE RESULTS OF MLP AND BASELINES UNDER PYGSD [42] SETUP ON EXISTENCE PREDICTION (EP) TASK

Method	Cora-ML	CiteSeer	Telegram	Cornell	Texas	Wisconsin
MLP	**78.85±0.83**	71.03±0.89	82.72±0.57	68.41±2.05	**69.58±2.29**	**72.53±2.27**
DGCN	76.45±0.49	70.72±0.79	83.18±1.55	67.95±2.27	63.65±2.40	68.12±2.86
DiGCN	76.17±0.49	72.00±0.88	83.12±0.43	67.16±1.82	63.54±3.85	67.01±2.47
DiGCNIB	78.80±0.51	**74.55±0.91**	84.49±0.51	**69.77±2.05**	67.60±3.44	70.78±2.79
MagNet	77.37±0.45	71.47±0.71	**85.82±0.39**	68.98±2.27	65.94±1.88	71.23±2.53

u two embedding vectors: a source embedding $\mathbf{s}_u$ and a target embedding $\mathbf{t}_u$. These embeddings are typically learned using either factorization-based or random walk-based techniques. Factorization-based methods include HOPE [6], which computes Katz similarity [29] followed by singular value decomposition (SVD) [30], and AROPE [8], which generalizes this approach to preserve arbitrary-order proximities. STRAP [9] extends this idea by combining Personalized PageRank (PPR) [2] scores from both the original and transposed graphs before applying SVD. Random walk-based methods include APP [7], which trains embeddings using PPR-guided random walks, and NERD [10], which samples nodes based on degree distributions. Other recent notable methods include DGGAN [11] (which employs adversarial training), ELTRA [12] (based on ranking-oriented learning), and ODIN [13] (which incorporates degree bias separation). All these methods focus on generating source–target embeddings from graph structures to support link prediction tasks.

B. Graph Neural Networks for Directed Graphs

Graph Neural Networks for directed graphs can be broadly classified into four categories based on their embedding strategies: (1) Source–target methods learn separate source and target embeddings for each node. DiGAE [14] applies GCN's convolutions [28] to both the adjacency matrix and its transpose. CoBA [15] jointly aggregates source and target neighbors, while BLADE [16] introduces an asymmetric loss to learn dual embeddings from local neighborhoods. (2) Gravity-inspired methods are motivated by Newton's law of universal gravitation, learning real-valued node embeddings along with a scalar mass parameter. Gravity GAE [17] combines a gravity-based decoder with a GCN-like encoder, and DHYPR [18] extends this approach through hyperbolic collaborative learning. (3) Single real-valued methods generate a single real-valued embedding per node. DGCN [19] constructs a directed Laplacian using first- and second-order proximities. MotifNet [31] employs motif-based Laplacians. DiGCN and DiGCNIB [20] generalize directed Laplacians using Personalized PageRank (PPR). DirGNN [21] introduces a flexible convolution framework for directed message-passing neural networks (MPNNs), and HoloNets [22] leverage holomorphic functional calculus with an architecture similar to DirGNN. (4) Complex-valued methods utilize Hermitian adjacency matrices to learn complex-valued embeddings. MagNet [24] defines a magnetic Laplacian to construct directed graph convolutions. LightDiC [25] scales this approach to large graphs via a decoupled design, while DUPLEX [26] employs dual GAT encoders with Hermitian adjacency matrices. Additional methods adapt Transformers to directed graphs. For example, DiGT [32] assigns each node separate source and target embeddings, and Geisler et al. [33] incorporate Magnetic Laplacian positional encodings. Related work also extends oversmoothing analysis to directed graphs [34]. While these methods propose various convolutional and propagation mechanisms for directed graphs, fair evaluation for directed link prediction tasks is often lacking. Many existing experimental setups omit key baselines or suffer from significant issues, such as label leakage. These challenges motivate this work.

C. Spectral-Based Graph Neural Networks

In recent years, spectral-based Graph Neural Networks (GNNs) have garnered significant attention and demonstrated strong performance across various tasks [35]. Many popular methods in this category approximate spectral graph filters using polynomials of the adjacency or Laplacian matrix [36], [37], [38], [39], [40]. Specifically, let $\mathbf{A}_{\mathrm{s}}$ denote the adjacency matrix of the derived undirected graph for $\mathcal{G}$, and let $\mathbf{D}_{\mathrm{s}}$ be its diagonal degree matrix, where $\mathbf{D}_{\mathrm{s}}[i,i] = \sum_j \mathbf{A}_{\mathrm{s}}[i,j]$. We define the symmetric normalized adjacency matrix as $\mathbf{P} = \mathbf{D}_{\mathrm{s}}^{-1/2}\mathbf{A}_{\mathrm{s}}\mathbf{D}_{\mathrm{s}}^{-1/2}$. The propagation process in spectral-based GNNs is then given by:

$$\mathbf{Z} = h(\mathbf{P})\mathbf{X} \approx \sum_{k=0}^{K} w_k \mathbf{P}^k \mathbf{X}, \tag{1}$$

where $h(\mathbf{P})$ represents the spectral graph filter and w_k are the polynomial coefficients. From a spectral perspective, we denote the eigendecomposition of the symmetric normalized adjacency matrix as $\mathbf{P} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top$, where $\mathbf{U}$ is the matrix of eigenvectors and $\mathbf{\Lambda}$ is the diagonal matrix of eigenvalues. Accordingly, the spectral graph filter can be expressed as $h(\mathbf{P}) = \mathbf{U}h(\mathbf{\Lambda})\mathbf{U}^\top$, meaning it operates as a function of the eigenvalues $\mathbf{\Lambda}$. Thus, spectral-based GNNs approximate the graph filter function $h(\mathbf{\Lambda})$ in the spectral domain using a polynomial expansion. Notably, spectral filters can also be equivalently defined using the Laplacian matrix. In this case, the propagation process becomes $\mathbf{Z} = \sum_{k=0}^{K} w_k \mathbf{L}^k \mathbf{X}$, where $\mathbf{L} = \mathbf{I} - \mathbf{P}$ is the normalized Laplacian matrix. Within this framework, many powerful spectral-based GNNs have been proposed. For example, ChebNet [36] uses Chebyshev polynomials to approximate spectral filters, while GCN [28] simplifies ChebNet to improve efficiency. More recently, GPR-GNN [37] learns the coefficients w_k directly, and both BernNet [38] and JacobiConv [39] use Bernstein and Jacobi polynomial bases, respectively. Despite their effectiveness, these methods are primarily designed for undirected graphs. The development of analogous approaches that leverage polynomial approximations of spectral filters for directed graphs remains a relatively unexplored area.

III. Rethinking Directed Link Prediction

In this section, we will revisit the link prediction task for directed graphs and introduce a unified framework to assess the expressiveness of existing methods. Meanwhile, we examine the current experimental setups for directed link prediction tasks and highlight four significant issues.

A. Unified Framework for Directed Link Prediction

The link prediction task on directed graphs is to predict potential directed links (edges) in an observed graph $\mathcal{G}'$, with the given structure of $\mathcal{G}'$ and node feature $\mathbf{X}$. Formally,

Definition 1: **Directed link prediction problem**. Given an observed graph $\mathcal{G}' = (V, E')$ and node feature $\mathbf{X}$, the goal of directed link prediction is to predict the likelihood of a directed edge $(u,v) \in E^\star$ existing, where $E^\star \subseteq (V \times V) \setminus E'$. The probability of edge (u,v) existing is given by

$$p(u,v) = f(u \to v \mid \mathcal{G}', \mathbf{X}). \tag{2}$$

The $f(\cdot)$ denotes a prediction model, such as embedding methods or graph neural networks. Unlike link prediction on undirected graphs [41], for directed graphs, it is necessary to account for directionality. Specifically, $p(u,v)$ and $p(v,u)$ are not equal; they represent the probability of a directed edge existing from node u to node v, and from node v to node u, respectively. To evaluate the expressiveness of existing methods for directed link prediction, we propose a unified framework:

$$(\boldsymbol{\theta}_u, \boldsymbol{\phi}_u) = \mathrm{Enc}(\mathcal{G}', \mathbf{X}, u), \quad \forall u \in V, \tag{3}$$

$$p(u,v) = \mathrm{Dec}(\boldsymbol{\theta}_u, \boldsymbol{\phi}_u, \boldsymbol{\theta}_v, \boldsymbol{\phi}_v), \quad \forall (u,v) \in E^\star. \tag{4}$$

Here, $\mathrm{Enc}(\cdot)$ represents an encoder function, which includes various methods described in Section I. And $\boldsymbol{\theta}_u \in \mathbb{R}^{d_\theta}$, $\boldsymbol{\phi}_u \in \mathbb{R}^{d_\phi}$ are real-valued dual embeddings of dimensions d_θ and d_ϕ, respectively. $\mathrm{Dec}(\cdot)$ is a decoder function tailored to the specific encoder method. This framework unifies existing methods for directed link prediction, as summarized in Table II. Specifically, here σ represents the activation function (e.g., Sigmoid), while MLP and LR denote the multilayer perceptron and the logistic regression predictor, respectively. The symbols $\odot$ and $\|$ represent the Hadamard product and the vector concatenation process, respectively. Then, we provide details on the embeddings and possible decoders used by the different methods listed in Table II.

- For single real-valued encoder, we define the real-valued embedding as $\mathbf{h}_u = \boldsymbol{\theta}_u$ and set $\boldsymbol{\phi}_u = \varnothing$, where $\varnothing$ denotes nonexistence. Possible decoders include: $\sigma\big(\mathbf{h}_u^\top \mathbf{h}_v\big)$, $\mathrm{MLP}\big(\mathbf{h}_u \odot \mathbf{h}_v\big)$, $\mathrm{MLP}\big(\mathbf{h}_u \| \mathbf{h}_v\big)$.
- For source-target encoder, we define the source embedding as $\mathbf{s}_u = \boldsymbol{\theta}_u$ and the target embedding as $\mathbf{t}_u = \boldsymbol{\phi}_u$ for each node $u \in V$. Possible decoders include: $\sigma\big(\mathbf{s}_u^\top \mathbf{t}_v\big)$, $\mathrm{MLP}\big(\mathbf{s}_u \odot \mathbf{t}_v\big)$, $\mathrm{LR}\big(\mathbf{s}_u \| \mathbf{t}_v\big)$, etc. Here, $\mathbf{s}_u^\top \mathbf{t}_v$ denotes the inner product of the source and target embeddings.
- For complex-valued encoder, we define the complex-valued embedding as $\mathbf{z}_u = \boldsymbol{\theta}_u \odot \exp(i\boldsymbol{\phi}_u)$, where i is the imaginary unit. Possible decoders include: $\mathrm{Direc}\big(\mathbf{z}_u, \mathbf{z}_v\big)$, $\mathrm{MLP}\big(\boldsymbol{\theta}_u \| \boldsymbol{\theta}_v \| \boldsymbol{\phi}_u \| \boldsymbol{\phi}_v\big)$, where $\mathrm{Direc}(\cdot)$ is a direction-aware function defined in DUPLEX [26].
- For gravity-inspired encoder, we define the real-valued embedding as $\mathbf{h}_u = \boldsymbol{\theta}_u$ and set the mass parameter $m_u = g(\boldsymbol{\phi}_u)$, where $g(\cdot)$ is a function or neural network that converts $\boldsymbol{\phi}_u$ into a scalar [18]. Possible decoders include: $\sigma\big(m_v - \lambda \log \|\mathbf{h}_u - \mathbf{h}_v\|_2^2\big)$ [17], $\sigma\big(m_v - \lambda \log\big(\mathrm{dist}(\mathbf{h}_u, \mathbf{h}_v)\big)\big)$ [18]. Here, λ is a hyperparameter and $\mathrm{dist}(\cdot)$ represents the hyperbolic distance.

Based on this framework, if $0 < d_\theta, d_\phi \ll n$, these encoders involve two real embeddings $\boldsymbol{\theta}_u$ and $\boldsymbol{\phi}_u$, which we refer to as dual methods. Conversely, if $d_\phi = 0$, these encoders have only a single real embedding $\boldsymbol{\theta}_u$, referred to as single methods. We analyse the expressiveness of dual and single methods in terms of asymmetry preservation and graph reconstruction.

Asymmetry preservation: Previous source-target encoders have demonstrated that using the source and target embeddings effectively preserves asymmetric information in directed graphs,

a capability that single methods lack [6], [7], [9], [12]. We extend this claim by arguing that all dual methods can preserve asymmetric information in directed graphs, including complex-valued and gravity-inspired encoders. Specifically, complex-valued methods [24], [26] encode directionality as a geometric difference in complex space, meaning that edges from node u to node v and edges from node v to node u can be distinguished by the difference between $\mathbf{z}_u^\top \overline{\mathbf{z}}_v$ and $\mathbf{z}_v^\top \overline{\mathbf{z}}_u$. The gravity-inspired methods [17], [18], on the other hand, use direction-sensitive gravity to preserve asymmetry, i.e., $Gm_u/||\mathbf{h}_u - \mathbf{h}_v||^2$ and $Gm_v/||\mathbf{h}_u - \mathbf{h}_v||^2$ distinguish between edges from node u to node v and edges from node v to node u, where G is the gravitational constant. Thus, in terms of preserving asymmetry in directed graphs, dual methods have a significant advantage over single methods, as the former can naturally preserve the asymmetry of directed edges, facilitating the link prediction task.

Graph reconstruction: Graph reconstruction involves using node embeddings $\{\boldsymbol{\theta}_u, \boldsymbol{\phi}_u\}$ and the decoder function $\text{Dec}(\cdot)$ to compute the probability $p(u, v)$ of each directed edge. The top-m edges are then selected to reconstruct the original graph. For accurate reconstruction, the encoder $\text{Enc}(\cdot)$ and decoder $\text{Dec}(\cdot)$ need to ensure $p(u, v) \to 1$ if an edge exists from u to v, and $p(u, v) \to 0$ otherwise. This task requires the model to capture structural properties of the graph, including edge existence and directionality, reflecting the expressiveness for the directed link prediction [9], [12]. Dual methods benefit from the theoretical advantage of asymmetry preservation. When equipped with a suitable $\text{Dec}(\cdot)$ function, these methods can achieve effective graph reconstruction. The underlying intuition is that: source-target methods can represent the neighbor matrix as $\mathbf{A}_{uv} = \mathbf{s}_u^\top \mathbf{t}_v$ [6], [9], [14], complex-valued methods can represent the Hermitian adjacency matrix as $\mathbf{H}_{uv} = \mathbf{z}_u^\top \overline{\mathbf{z}}_v$ [24], [26], and gravity-inspired methods can represent the neighbor matrix as $\mathbf{A}_{uv} = Gm_u/||\mathbf{h}_u - \mathbf{h}_v||^2$ [17], [18]. In contrast, single methods lack the intrinsic preservation of asymmetry, but they can partially capture the graph structure by using asymmetric decoders, such as $\text{MLP}(\mathbf{h}_u \| \mathbf{h}_v)$. While this allows for limited structural reconstruction, single methods are theoretically insufficient to represent arbitrary directed graphs. As formalized in Proposition 2 (see the Appendix for the proof available online), they fail to reconstruct certain structures, such as directed ring graphs. Therefore, although asymmetric decoders enhance the performance of single methods, their overall expressiveness remains limited compared to dual methods.

Proposition 2: Single methods (single real-valued embedding $\mathbf{h}_u$) with an asymmetric decoder function $\text{MLP}(\mathbf{h}_u \| \mathbf{h}_v)$ can capture graph structure and enable reconstruction for some specific directed graphs, but not arbitrary directed graphs, such as directed ring graphs.

Overall, dual methods have a clear theoretical advantage in both asymmetry preservation and graph reconstruction, making them more expressive for link prediction compared to single methods. However, we also want to highlight the critical role that decoder function design plays in link prediction tasks. While single embeddings are inherently limited, they can still benefit from asymmetric decoders in practical applications. These theoretical insights motivate a deeper empirical analysis of how different encoder and decoder functions impact performance in link prediction tasks.

B. Issues With Existing Experimental Setups

The existing experimental setups for directed link prediction are generally divided into two categories. The first is the multiple subtask setup, which includes tasks such as existence prediction (EP), direction prediction (DP), three-class prediction (3 C), and four-class prediction (4 C). This approach treats directed link prediction as a multi-class classification problem, where the model must predict edges as positive (original direction), inverse (reverse direction), bidirectional (both directions), or nonexistent (no connection). Specifically:

- EP: The model predicts whether a directed edge (u, v) exists in the graph, treating both reverse and nonexistent edges as nonexistent.
- DP: The model predicts the direction of edges for node pairs (u, v), where either $(u, v) \in E$ or $(v, u) \in E$.
- 3 C: The model classifies an edge as positive, reverse, or nonexistent.
- 4 C: The model classifies edges into four classes: positive, reverse, bidirectional, or nonexistent.

The multiple subtask setup is widely adopted by existing GNN methods [24], [25], [26], [42], [43], [44]. The second category is the non-standardized setup defined in various papers [9], [13], [14], [15], [18]. These setups involve different datasets, inconsistent splitting strategies, and varying evaluation metrics. Below, we discuss the four significant issues with existing setups.

Issue 1. The Multi-layer Perceptron (MLP) is a neglected but powerful baseline: Most existing setups fail to report the performance of MLP. We evaluate MLP across three popular multiple subtask setups: MagNet [24], PyGSD [42], and DUPLEX [26], which cover a variety of datasets and baselines. Fig. 1(a) and 1(b) present the results of our reproduced MagNet experiments alongside the MLP performance, showing that the MLP performs comparably to MagNet on the DP and EP tasks. Tables III and IV display MLP results alongside several other baselines on the DP and EP tasks within the PyGSD setup, with the highest values highlighted in bold. Across six datasets, MLP demonstrates competitive performance, achieving state-of-the-art (SOTA) results for both tasks on the Texas and Wisconsin datasets. Tables V and VI present the replicated DUPLEX experiments and MLP results on the Cora and CiteSeer datasets, showing that MLP achieves competitive performance. Interestingly, despite DUPLEX being a recent advancement in directed graph learning, MLP outperforms it on certain tasks. These findings highlight the lack of fundamental baselines in previous studies and suggest that current benchmarks do not provide a sufficient challenge. More importantly, the results indicate that MLP has achieved SOTA performance across various setups and datasets, challenging the conclusions of prior studies and contradicting the theoretical assumption that dual methods are more expressive.

Issue 2. Many benchmarks suffer from label leakage: As defined in Definition 1, directed link prediction aims to predict potential edges from observed graphs, with the key principle

TABLE V
LINK PREDICTION RESULTS ON CORA DATASET UNDER THE DUPLEX [26] SETUP: RESULTS WITHOUT SUPERSCRIPTS ARE FROM THE DUPLEX PAPER, † INDICATES REPRODUCTION WITH TEST SET EDGES IN TRAINING, AND ‡ INDICATES REPRODUCTION WITHOUT TEST SET EDGES IN TRAINING

Method	EP(ACC)	EP(AUC)	DP(ACC)	DP(AUC)	3C(ACC)	4C(ACC)
MagNet	81.4±0.3	89.4±0.1	88.9±0.4	95.4±0.2	66.8±0.3	63.0±0.3
DUPLEX	93.2±0.1	95.9±0.1	95.9±0.1	97.9±0.2	92.2±0.1	88.4±0.4
DUPLEX†	93.49±0.21	95.61±0.20	95.25±0.16	96.34±0.23	92.41±0.21	89.76±0.25
MLP†	88.53±0.22	**95.46±0.18**	**95.76±0.21**	**99.25±0.06**	79.97±0.48	78.49±0.26
DUPLEX‡	87.43±0.20	91.16±0.24	88.43±0.16	91.74±0.38	84.53±0.34	81.36±0.46
MLP‡	84.00±0.29	**91.52±0.25**	**90.83±0.16**	**96.48±0.28**	72.93±0.21	71.51±0.20

TABLE VI
LINK PREDICTION RESULTS FOR CITESEER DATASET UNDER THE DUPLEX [26] SETUP: RESULTS WITHOUT SUPERSCRIPTS ARE FROM THE DUPLEX PAPER, † INDICATES REPRODUCTION WITH TEST SET EDGES IN TRAINING, AND ‡ INDICATES REPRODUCTION WITHOUT TEST SET EDGES IN TRAINING

Method	EP(ACC)	EP(AUC)	DP(ACC)	DP(AUC)	3C(ACC)	4C(ACC)
MagNet	80.7±0.8	88.3±0.4	91.7±0.9	96.4±0.6	72.0±0.9	69.3±0.4
DUPLEX	95.7±0.5	98.6±0.4	98.7±0.4	99.7±0.2	94.8±0.2	91.1±1.0
DUPLEX†	92.11±0.78	95.85±0.87	97.54±0.54	98.93±0.59	88.22±1.06	84.77±1.01
MLP†	85.74±1.80	93.33±1.27	**97.55±0.97**	**99.58±0.52**	81.20±0.82	76.30±0.62
DUPLEX‡	83.59±1.47	89.34±1.03	85.56±1.36	91.82±0.98	76.37±2.07	73.80±2.01
MLP‡	77.34±1.86	87.36±1.26	**89.19±0.95**	**95.97±0.61**	67.82±1.21	64.25±1.35

that test edges must remain hidden during training to avoid label leakage. However, some current setups violate this principle. For example, 1) MagNet [24], PyGSD [42], and DUPLEX [26] expose test edges during negative edge sampling in the training process, indirectly revealing the test edges' presence to the model. 2) LightDiC [25] uses eigenvectors of the Laplacian matrix of the entire graph as input features, embedding test edge information in the training input. 3) DUPLEX propagates information across the entire graph during training, making the test edges directly visible to the model. To investigate, we experiment with DUPLEX using its original code. As shown in Tables V and VI, DUPLEX† (original settings with label leakage) clearly outperforms DUPLEX‡ (propagation restricted to training edges) due to label leakage. Similar results are observed with MLP: MLP† (using in/out degrees from test edges) significantly outperforms MLP‡ (using only training-edge in/out degrees). These findings underscore that even the leakage of degree information can significantly impact performance.

Issue 3. Multiple subtask setups result in class imbalances and limited evaluation metrics: The multiple subtask setups treat the directed link prediction task as a multi-class classification problem, causing significant class imbalances that hinder model training. For example, the 4 C task in DUPLEX classifies edges into reverse, positive, bidirectional, and nonexistent. However, bidirectional edges are rare in real-world directed graphs, and reverse edges are often assigned arbitrarily, lacking meaningful semantic interpretation. Fig. 2(a) and 2(b) illustrate this imbalance, highlighting the difficulty of accurately predicting bidirectional and nonexistent edges on the Cora and CiteSeer datasets. Additionally, these setups rely heavily on accuracy as the evaluation metric, which provides a limited and potentially misleading assessment of model performance. Given the nature of link prediction, ranking-based metrics such as Hits@K and Mean Reciprocal Rank (MRR) are more appropriate, a perspective well-established in evaluating undirected link prediction methods [45].

Issue 4. Lack of standardization in dataset splits and feature inputs: Current settings face inconsistent dataset splits. In multiple subtask setups, edges are typically split into 80% for training, 5% for validation, and 15% for testing. However, class proportions are further manually adjusted for balance, which leads to varying training and testing ratios across different datasets. Non-standardized setups are more confusing, for example, ELTRA [12] uses 90% of edges for training, STRAP [9] uses 50%, and DiGAE [14] uses 85%, making cross-study results difficult to evaluate. Feature input standards are also lacking. Embedding methods often omit node features, while GNNs require them. MagNet and PyGSD use in/out degrees, DUPLEX uses random normal distributions, LightDiC uses original features or Laplacian eigenvectors, and DHYPR [18] uses identity matrices. This inconsistency undermines reproducibility. As shown in Tables III, IV, V, VI, and Fig. 1, reproduced results often deviate significantly, with some better and others worse.

In the above experiments, we strictly follow the configurations of each setup and reproduce the results using the provided codes and datasets. For the MLP model, we implement a simple two-layer network with 64 hidden units, tuning the learning rate and weight decay to match each setup for a fair comparison. These issues highlight the need for a new benchmark in directed link prediction, enabling fair evaluation and supporting future research.

IV. NEW BENCHMARK: DIRLINKBENCH

In this section, we introduce a new robust benchmark for the directed link prediction tasks, **DirLinkBench**, which offers three key advantages:

TABLE VII
STATISTICS OF DIRLINKBENCH DATASETS

Datasets	#Nodes	#Edges	Avg. Degree	#Features	%Directed Edges	Description
Cora-ML	2,810	8,229	5.9	2,879	93.97	citation network
CiteSeer	2,110	3,705	3.5	3,703	98.00	citation network
Photo	7,487	143,590	38.4	745	65.81	co-purchasing network
Computers	13,381	287,076	42.9	767	71.23	co-purchasing network
WikiCS	11,311	290,447	51.3	300	48.43	weblink network
Slashdot	74,444	424,557	11.4	-	80.17	social network
Epinions	100,751	708,715	14.1	-	65.04	social network

- *Comprehensive coverage:* DirLinkBench includes seven real-world datasets, 16 baselines, and seven evaluation metrics. These datasets span diverse domains, scales, and structural properties, and are uniformly preprocessed to support consistent evaluation. The baselines cover both embedding methods and GNNs under fair settings. Notably, to the best of our knowledge, DirLinkBench is the first to introduce ranking-based metrics for evaluating directed link prediction.
- *Standardized evaluation:* DirLinkBench addresses the issues of existing benchmarks discussed in Section III-B by redesigning the task setup. It establishes a unified framework for dataset splitting, feature initialization, and evaluation metrics, ensuring fairness, consistency, and reproducibility across models.
- *Modular extensibility:* Built on PyTorch Geometric (PyG) [46], DirLinkBench is highly modular and extensible, facilitating the integration of new datasets, model architectures, and configurable components such as feature initialization strategies and negative sampling schemes.

Next, we provide a detailed overview of DirLinkBench, covering its datasets, task setup, baseline methods, and results.

A. Dataset

DirLinkBench comprises seven real-world directed graphs from diverse domains. Specifically, the datasets include: 1) Two citation networks, Cora-ML [47], [48] and CiteSeer [49], where nodes represent academic papers and directed edges denote citation relationships. 2) Two Amazon co-purchasing networks, Photo and Computers [50], in which nodes denote products and directed edges represent sequential purchase behavior. 3) WikiCS [51], a weblink network where nodes correspond to computer science articles on Wikipedia and directed edges indicate hyperlinks between articles. 4) Two social networks, Slashdot [52] and Epinions [53]. In Slashdot, nodes represent users and directed edges capture explicit social interactions such as friendships or replies, while in Epinions, directed edges encode trust relationships, offering a view into user-to-user reliability assessments.

These datasets are publicly available and have been widely used in tasks such as node classification [54] and link prediction [42]. However, many of them contain noise, such as duplicate edges, self-loops, and isolated nodes, that can negatively impact the evaluation of link prediction methods. To address this, we preprocess the datasets by removing duplicate edges and self-loops. Following the protocols in [50], [54], we also eliminate isolated nodes and retain only the largest connected component to ensure standardized evaluation conditions for link prediction. We summarize the statistical characteristics of the datasets in Table VII, where Avg. Degree indicates average node connectivity, and %Directed Edges reflects inherent directionality.

The datasets not only originate from different domains but also vary significantly in size. For example, Epinions contains over 700,000 edges and more than 100,000 nodes. They also differ in average node degree; CiteSeer has the lowest average degree (3.5), while WikiCS has the highest (51.3). Additionally, all datasets include original node features except Slashdot and Epinions. Compared to previous benchmarks, our collection spans multiple domains, dataset sizes, structural characteristics, and standardized pre-processing, enabling a more comprehensive evaluation of link prediction methods across diverse real-world scenarios.

B. Task Setup

In Section III-B, we discussed the issues of existing benchmarks, which mainly stem from their task setup. DirLinkBench addresses these issues through the redesigned task setup. First, we argue that the commonly used multiple subtask setup is flawed and inadequate for evaluating directed link prediction methods. In Issue 1, we demonstrate that under this setup, a simple MLP achieves unexpectedly high accuracy (80–90%) across several datasets, surpassing SOTA methods in some cases. This counterintuitive result highlights a fundamental weakness of the multiple subtask setup—it fails to distinguish between simplistic baselines and specialized approaches. Furthermore, in Issue 3, we show that this setup inherently introduces class imbalance and limits the applicability of ranking-based metrics. DirLinkBench adopts a binary classification task for directed link prediction to address these limitations: given two nodes u and v, the task is to predict whether a directed edge exists from u to v. Notably, this setup aligns with the directed link prediction problem definition in Definition 1. Moreover, this setting has been widely adopted in prior studies on embedding methods [9], [12], and its extension to GNNs is straightforward and intuitive.

Second, to address label leakage (Issue 2) and unstandardized data splits (Issue 4), DirLinkBench adopts a standardized task setup. Specifically, given a directed graph $\mathcal{G}$, we randomly split

15% of the edges for testing, 5% for validation, and use the remaining 80% for training, while ensuring that the training graph $\mathcal{G}'$ remains weakly connected [42]. For testing and validation, we sample an equal number of negative edges under the full graph $\mathcal{G}$ visible, while for training, only the training graph $\mathcal{G}'$ is accessible. Feature inputs are provided in three forms: 1) original node features, 2) in/out degrees computed from the training graph $\mathcal{G}'$ [24], or 3) random feature vectors sampled from a standard normal distribution [26]. For fair comparisons, we generate 10 fixed splits using random seeds, and all models are evaluated on shared splits to report the mean performance. Each model learns from the training graph and feature inputs to compute the probability $p(u, v)$ of test edges.

C. Baseline

We carefully select 15 state-of-the-art baselines, including three embedding methods: STRAP [9], ODIN [13], ELTRA [12]; a basic method MLP; four classic undirected GNNs: GCN [28], GAT [55], APPNP [54], GPRGNN [37]; four single real-valued methods: DGCN [19], DiGCN [20], DiGCNIB [20], DirGNN [21]; two complex-valued methods: MagNet [24], DUPLEX [26]; a gravity-inspired method: DHYPR [18]; and a source-target GNN: DiGAE [14]. Note that some recent methods, such as CoBA [15], BLADE [16], and NDDGNN [23], are not included due to unavailable code.

For baseline implementation, we use the original code released by the authors or widely adopted libraries such as PyTorch Geometric (PyG) [46] and PyTorch Geometric Signed Directed (PyGSD) [42]. For methods without publicly available link prediction code (e.g., GCN, DGCN), we implement a variety of decoders and loss functions. For methods with official link prediction code (e.g., MagNet, DiGAE), we strictly follow the authors' reported settings. Hyperparameters are tuned via grid search, adhering to the configurations specified in each paper. Specifically, for STRAP, ODIN, and ELTRA, we consider four decoder options: $\sigma\left(\mathbf{s}_u^\top \mathbf{t}_v\right)$, $\mathrm{LR}\left(\mathbf{s}_u \odot \mathbf{t}_v\right)$, $\mathrm{LR}\left(\mathbf{s}_u \| \mathbf{t}_v\right)$, and $\mathrm{LR}\left(\mathbf{s}_u \| \mathbf{s}_v \| \mathbf{t}_u \| \mathbf{t}_v\right)$ [13]. Embedding generation for these methods also follows the parameter settings provided in their respective papers. For MLP, GCN, GAT, APPNP, GPRGNN, DGCN, DiGCN, DiGCN-IB, and DirGNN, we consider two commonly used loss functions: cross-entropy (CE) loss [24], [26] and binary cross-entropy (BCE) loss [14], [18], and three decoders: $\mathrm{MLP}(\mathbf{h}_u \| \mathbf{h}_v)$, $\mathrm{MLP}(\mathbf{h}_u \odot \mathbf{h}_v)$, and $\sigma(\mathbf{h}_u^\top \mathbf{h}_v)$. For MagNet, DUPLEX, DHYPR, and DiGAE, we adopt the loss functions and decoders as reported in their original implementations. For all methods, we train for up to 2000 epochs with an early stopping criterion of 200 epochs. We repeat each experiment 10 times and report the mean and standard deviation. More detailed hyperparameters for each baseline are provided in the Appendix, available online.

D. Result

For evaluation, we introduce ranking-based metrics to directed link prediction benchmarks for the first time, including Hits@20, Hits@50, Hits@100, and MRR, which are widely used in undirected link prediction benchmarks [45], [56]. In addition, we report AUC, AP, and ACC for comparative purposes. A detailed description of all metrics is provided in the Appendix, available online. For each method, we report the best mean result across different combinations of feature inputs, loss functions, and decoders. Table VIII shows the results under the Hits@100 metric, while Table IX shows the results across all metrics for some datasets. Complete results for all metrics are included in the code repository due to space constraints.

The results reveal that ACC, AUC, and AP offer limited discriminatory ability across different baselines. For example, some simple undirected GNNs (e.g., GAT, APPNP) achieve competitive performance, with only minor performance gaps across methods. Conversely, methods that perform well under ranking-based metrics (e.g., STRAP, DiGAE) tend to perform relatively poorly. These findings suggest that ACC, AUC, and AP are inadequate for reliably evaluating directed link prediction performance, aligning with recent discussions about their limitations even in undirected settings [45], [57]. Therefore, we argue that ranking-based metrics are better suited for the link prediction task. Since Hits@100 is widely used in popular benchmarks (e.g., Open Graph Benchmark (OGB) [56]) and reveals significant performance differences among methods, we adopt it as our primary evaluation metric.

From the results in Table VIII, we observe that embedding methods maintain a strong advantage, even without feature inputs. Early single real-valued undirected and directed GNNs also demonstrate competitive performance. In contrast, several newer directed GNNs (e.g., MagNet [24], DUPLEX [26], DHYPR [18]) exhibit weaker performance or face scalability challenges. We provide a deeper analysis of these observations in Section VI. Interestingly, the simple directed graph autoencoder DiGAE [14] is the top-performing method overall, but it underperforms on specific datasets (e.g., Cora-ML, CiteSeer). This finding motivates us to revisit DiGAE's design and propose improved methods to enhance directed link prediction.

V. New Method: SDGAE

In this section, we revisit DiGAE's model architecture to better understand its encoder's graph convolution mechanism. We then introduce a novel method: Spectral Directed Graph Auto-Encoder (SDGAE).

A. Understand the Graph Convolution of DiGAE

DiGAE [14] is a graph auto-encoder designed for directed graphs. Its encoder's graph convolutional layer is denoted as

$$\mathbf{S}^{(\ell+1)} = \sigma\left(\hat{\mathbf{D}}_{\text{out}}^{-\beta} \hat{\mathbf{A}} \hat{\mathbf{D}}_{\text{in}}^{-\alpha} \mathbf{T}^{(\ell)} \mathbf{W}_T^{(\ell)}\right), \tag{5}$$

$$\mathbf{T}^{(\ell+1)} = \sigma\left(\hat{\mathbf{D}}_{\text{in}}^{-\alpha} \hat{\mathbf{A}}^\top \hat{\mathbf{D}}_{\text{out}}^{-\beta} \mathbf{S}^{(\ell)} \mathbf{W}_S^{(\ell)}\right). \tag{6}$$

Here, $\hat{\mathbf{A}} = \mathbf{A} + \mathbf{I}$ denotes the adjacency matrix with added self-loops, and $\hat{\mathbf{D}}_{\text{out}}$ and $\hat{\mathbf{D}}_{\text{in}}$ represent the corresponding out-degree and in-degree matrices, respectively. $\mathbf{S}^{(\ell)}$ and $\mathbf{T}^{(\ell)}$ denote the source and target embeddings at the ℓ-th layer, initialized as $\mathbf{S}^{(0)} = \mathbf{T}^{(0)} = \mathbf{X}$. The hyperparameters α and β are degree-based normalization factors, σ is the activation function (e.g.,

TABLE VIII
PERFORMANCE UNDER THE HITS@100 METRIC. THE BEST AND SECOND-BEST RESULTS ARE HIGHLIGHTED IN **BOLD** AND UNDERLINE, RESPECTIVELY. "TO" INDICATES METHODS THAT DID NOT COMPLETE WITHIN 24 HOURS, AND "OOM" INDICATES OUT-OF-MEMORY ERRORS

Method	Cora-ML	CiteSeer	Photo	Computers	WikiCS	Slashdot	Epinions	Avg. Rank ↓
STRAP	79.09±1.57	69.32±1.29	**69.16±1.44**	51.87±2.07	**76.27±0.92**	31.43±1.21	**58.99±0.82**	5.57
ODIN	54.85±2.53	63.95±2.98	14.13±1.92	12.98±1.47	9.83±0.47	34.17±1.19	36.91±0.47	13.57
ELTRA	87.45±1.48	84.97±1.90	20.63±1.93	14.74±1.55	9.88±0.70	33.44±1.00	41.63±2.53	9.29
MLP	60.61±6.64	70.27±3.40	20.91±4.18	17.57±0.85	12.99±0.68	32.97±0.51	44.59±1.62	11.43
GCN	70.15±3.01	80.36±3.07	58.77±2.96	43.77±1.75	38.37±1.51	33.16±1.22	46.10±1.37	6.14
GAT	79.72±3.07	85.88±4.98	58.06±4.03	40.74±3.22	40.47±4.10	30.16±3.11	43.65±4.88	6.29
APPNP	86.02±2.88	83.57±4.90	47.51±2.51	32.24±1.40	20.23±1.72	33.76±1.05	41.99±1.23	8.14
GPRGNN	86.03±2.73	88.70±2.96	47.60±5.09	38.39±2.64	20.87±3.15	32.61±1.05	41.14±2.10	7.43
DGCN	63.32±2.59	68.97±3.39	51.61±6.33	39.92±1.94	25.91±4.10	TO	TO	10.86
DiGCN	63.21±5.72	70.95±4.67	40.17±2.38	27.51±1.67	25.31±1.84	TO	TO	12.14
DiGCNIB	80.57±3.21	85.32±3.70	48.26±3.98	32.44±1.85	28.28±2.44	TO	TO	9.14
DirGNN	76.13±2.85	76.83±4.24	49.15±3.62	35.65±1.30	50.48±0.85	41.74±1.15	50.10±2.06	6.43
MagNet	56.54±2.95	65.32±3.26	13.89±0.32	12.85±0.59	10.81±0.46	31.98±1.06	28.01±1.72	14.29
DUPLEX	69.00±2.52	73.39±3.42	17.94±0.66	17.90±0.71	8.52±0.60	18.42±2.59	16.50±4.34	13.14
DHYPR	86.81±1.60	92.32±3.72	20.93±2.41	TO	TO	OOM/TO	OOM/TO	11.29
DiGAE	82.06±2.51	83.64±3.21	55.05±2.36	41.55±1.62	29.21±1.36	41.95±0.93	55.14±1.96	4.71
SDGAE	**90.37±1.33**	**93.69±3.68**	68.84±2.35	**53.79±1.56**	54.67±2.50	**42.42±1.15**	55.91±1.77	**1.43**

TABLE IX
PERFORMANCE ACROSS SEVEN DIFFERENT METRICS ON CORA-ML, PHOTO, AND SLASHDOT DATASETS. THE BEST AND SECOND-BEST RESULTS ARE HIGHLIGHTED IN **BOLD** AND UNDERLINE, RESPECTIVELY

Dataset	Method	Hits@20	Hits@50	Hits@100	MRR	AUC	AP	ACC
Cora-ML	STRAP	67.10±2.55	75.05±1.66	79.09±1.57	**30.91±7.48**	87.38±0.83	90.46±0.70	78.47±0.77
	ELTRA	70.74±5.47	81.93±2.38	87.45±1.48	19.77±5.22	94.83±0.58	95.37±0.52	85.40±0.45
	MLP	29.84±4.83	44.28±5.72	60.61±6.64	11.84±3.40	89.93±2.09	88.55±2.51	81.32±2.41
	GAT	33.42±8.93	58.40±6.55	79.72±3.07	11.07±3.68	94.57±0.45	92.77±0.42	88.95±0.71
	APPNP	55.47±4.63	75.16±4.09	86.02±2.88	22.49±6.33	95.94±0.57	95.82±0.59	89.90±0.73
	DiGCNIB	45.90±4.97	66.62±3.67	80.57±3.21	17.08±3.90	94.67±0.56	94.21±0.65	87.25±0.58
	DirGNN	42.48±7.34	59.41±4.00	76.13±2.85	13.34±5.56	93.05±0.87	92.52±0.79	85.83±0.93
	MagNet	29.38±3.39	43.28±3.76	56.54±2.95	11.19±3.04	85.23±0.84	86.06±0.94	77.51±0.92
	DUPLEX	21.73±3.19	34.98±2.37	69.00±2.52	7.74±1.95	88.02±0.95	86.62±1.43	82.28±0.93
	DiGAE	56.13±3.80	72.23±2.51	82.06±2.51	20.53±4.21	92.56±0.66	93.70±0.54	86.25±0.80
	SDGAE	**70.89±3.35**	**83.63±2.15**	**90.37±1.33**	28.45±5.82	**97.24±0.34**	**97.21±0.17**	**91.36±0.70**
Photo	STRAP	38.54±5.20	55.21±2.19	**69.16±1.44**	12.08±3.15	98.54±0.04	98.65±0.05	94.61±0.10
	ELTRA	7.09±1.23	12.86±1.58	20.63±1.93	2.22±0.59	96.89±0.06	95.84±0.13	91.84±0.12
	MLP	8.83±2.06	14.07±2.87	20.91±4.18	3.18±1.06	95.29±0.37	93.60±1.06	88.31±0.50
	GAT	25.97±4.22	42.85±4.90	58.06±4.03	8.62±3.07	99.13±0.09	98.93±0.11	**96.17±0.20**
	APPNP	22.13±2.60	35.30±2.04	47.51±2.51	6.56±1.14	98.54±0.09	98.26±0.12	94.71±0.29
	DiGCNIB	21.42±2.77	34.97±2.67	48.26±3.98	6.82±1.53	98.67±0.14	98.39±0.16	95.05±0.27
	DirGNN	22.59±2.77	34.65±3.31	49.15±3.62	8.72±2.08	98.76±0.09	98.47±0.13	95.34±0.17
	MagNet	5.35±0.50	9.04±0.52	13.89±0.32	1.62±0.39	88.11±0.21	87.48±0.21	80.31±0.14
	DUPLEX	7.84±1.17	12.64±1.22	17.94±0.66	2.53±0.66	94.22±0.76	93.37±0.91	87.68±0.52
	DiGAE	27.79±3.85	43.32±3.36	55.05±2.36	9.38±2.47	97.98±0.08	97.99±0.10	91.77±0.18
	SDGAE	**40.89±3.86**	**55.76±4.08**	68.84±2.35	**14.82±4.22**	**99.25±0.05**	**99.16±0.06**	96.16±0.14
Slashdot	STRAP	19.10±1.06	25.39±1.43	31.43±1.21	**9.82±1.82**	94.74±0.05	95.13±0.05	88.19±0.08
	ELTRA	18.02±2.11	26.31±0.95	33.44±1.00	5.53±1.77	94.65±0.03	95.23±0.04	88.11±0.11
	MLP	14.16±5.22	24.01±0.79	32.97±0.51	4.14±1.71	95.84±0.07	96.21±0.05	89.62±0.11
	GAT	14.82±2.70	22.19±2.78	30.16±3.11	5.11±1.53	96.26±0.18	96.45±0.20	88.01±0.49
	APPNP	15.00±5.47	24.34±1.47	33.76±1.05	5.86±2.78	96.21±0.06	96.43±0.05	90.07±0.10
	DirGNN	20.55±2.85	31.20±1.18	41.74±1.15	7.52±3.24	**96.95±0.05**	**97.14±0.06**	**90.65±0.13**
	MagNet	12.55±0.75	22.34±0.42	31.98±1.06	2.83±0.51	96.57±0.09	96.69±0.10	90.45±0.09
	DUPLEX	5.67±1.85	11.49±3.36	18.42±2.59	1.81±0.83	94.36±3.25	94.48±2.76	85.42±3.42
	DiGAE	**23.68±0.94**	**33.97±1.06**	41.95±0.93	5.54±1.51	95.26±0.29	96.13±0.19	85.67±0.28
	SDGAE	23.57±2.11	33.75±1.48	**42.42±1.15**	8.41±3.80	96.70±0.10	97.06±0.08	**91.05±0.20**

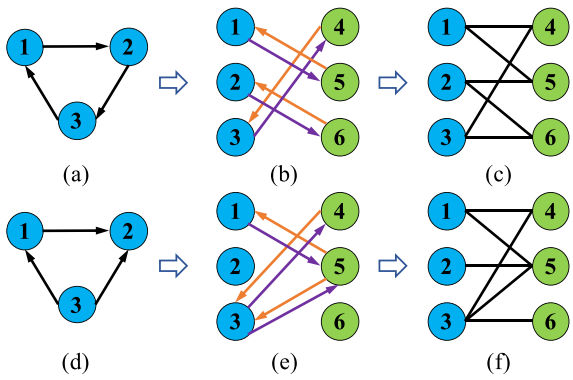


Fig. 3. The bipartite graph representation of two toy directed graphs.

ReLU), and $\mathbf{W}_T^{(\ell)}, \mathbf{W}_S^{(\ell)}$ represent the learnable weight matrices. The design of this graph convolution is inspired by the connection between GCN [28] and the 1-WL [58] algorithm for directed graphs. However, the underlying principles of this convolution remain unexplored, leaving its meaning unclear. And, the heuristic hyperparameters α and β introduce significant challenges for effective parameter optimization.

We revisit the graph convolution of DiGAE's encoder and observe that it corresponds to the GCN convolution [28] applied to an undirected bipartite graph. Specifically, given a directed graph $\mathcal{G}$ with adjacency matrix $\mathbf{A}$, the adjacency matrix of its bipartite representation [59] is defined as:

$$\mathcal{S}(\hat{\mathbf{A}}) := \begin{bmatrix} \mathbf{0} & \hat{\mathbf{A}} \\ \hat{\mathbf{A}}^\top & \mathbf{0} \end{bmatrix} \in \mathbb{R}^{2n \times 2n}. \tag{7}$$

Here, $\mathcal{S}(\hat{\mathbf{A}})$ is a block matrix consisting of $\hat{\mathbf{A}}$ and $\hat{\mathbf{A}}^\top$. It is evident that $\mathcal{S}(\hat{\mathbf{A}})$ represents the adjacency matrix of an undirected bipartite graph with $2n$ nodes. Fig. 3 provides two toy examples of directed graphs and their corresponding undirected bipartite graph representations. Notably, the self-loops in $\hat{\mathbf{A}}$ serve a fundamentally different purpose than in GCN [28]. In this context, the self-loops ensure the connectivity of the undirected bipartite graph. Without the self-loops, as illustrated in graphs (b) and (e) of Fig. 3, the graph structure suffers from significant connectivity issues. Based on these insights, we present the following Lemma 3.

Lemma 3: If omitting degree-based normalization in (5) and (6), the graph convolution of DiGAE's encoder is

$$\left[\mathbf{S}^{(\ell+1)}, \mathbf{T}^{(\ell+1)}\right]^\top = \sigma\left(\mathcal{S}(\hat{\mathbf{A}})\left[\mathbf{S}^{(\ell)}\mathbf{W}_S^{(\ell)}, \mathbf{T}^{(\ell)}\mathbf{W}_T^{(\ell)}\right]^\top\right). \tag{8}$$

The proof and its extension with degree-based normalization are provided in the Appendix, available online. Lemma 3 and the extension in its proof reveal that the graph convolution of DiGAE's encoder essentially corresponds to the GCN convolution applied to an undirected bipartite graph.

B. Spectral Directed Graph Auto-Encoder (SDGAE)

Building on the understanding of DiGAE, we identify its three key limitations: 1) DiGAE struggles to use deep networks for capturing rich structural information due to excessive learnable weight matrices, a problem also existing in deep GCN [60]. We provide experimental evidence for this in Section VI-A. 2) From the spectral perspective, DiGAE uses a fixed low-pass filter and cannot learn arbitrary spectral graph filters [61], similar to GCN, since both rely on the same convolutional formulation. 3) DiGAE relies on heuristic hyperparameters α and β, which are difficult to tune effectively.

To overcome these limitations, we propose SDGAE, which uses a polynomial approximation to learn arbitrary graph filters for directed graphs. SDGAE is inspired by spectral-based undirected GNNs [37], [38] and incorporates symmetric normalization of the directed adjacency matrix. The propagation process of SDGAE's encoder is defined as follows:

$$\begin{bmatrix} \mathbf{S}^{(K)} \\ \mathbf{T}^{(K)} \end{bmatrix} = \sum_{k=0}^{K} \mathbf{W}^{(k)} \begin{bmatrix} \mathbf{0} & \tilde{\mathbf{A}} \\ \tilde{\mathbf{A}}^\top & \mathbf{0} \end{bmatrix}^k \begin{bmatrix} \mathbf{S}^{(0)} \\ \mathbf{T}^{(0)} \end{bmatrix}. \tag{9}$$

Here, $\mathbf{W}^{(k)} = \text{diag}([w_S^{(k)}\mathbf{I}_n, w_T^{(k)}\mathbf{I}_n]) \in \mathbb{R}^{2n \times 2n}$ represents the diagonal coefficients matrix and $w_S^{(k)}, w_T^{(k)} \in \mathbb{R}$ are the corresponding polynomial coefficients. The parameter $K \in \mathbb{N}^+$ is the polynomial order and $\tilde{\mathbf{A}} = \hat{\mathbf{D}}_{\text{out}}^{-1/2}\hat{\mathbf{A}}\hat{\mathbf{D}}_{\text{in}}^{-1/2}$ denotes the normalization of the directed adjacency matrix $\hat{\mathbf{A}}$. Below Lemma 4 (see the Appendix for the proof) shows that this normalization is equivalent to the symmetric normalization of $\mathcal{S}(\hat{\mathbf{A}})$. In this Equation, $\mathbf{S}^{(0)}$ and $\mathbf{T}^{(0)}$ represent the initial source and target embeddings, while $\mathbf{S}^{(K)}$ and $\mathbf{T}^{(K)}$ denote the corresponding embeddings after K propagation steps.

Lemma 4: The symmetrically normalized block adjacency matrix $\mathbf{D}_{\mathcal{S}}^{-1/2}\mathcal{S}(\hat{\mathbf{A}})\mathbf{D}_{\mathcal{S}}^{-1/2} = \mathcal{S}(\tilde{\mathbf{A}})$, where $\mathbf{D}_{\mathcal{S}} = \text{diag}(\hat{\mathbf{D}}_{\text{out}}, \hat{\mathbf{D}}_{\text{in}})$ is the diagonal degree matrix of $\mathcal{S}(\hat{\mathbf{A}})$.

Spectral Analysis: Intuitively, SDGAE's encoder is a polynomial spectral-based GNN defined on an undirected bipartite graph with $2n$ nodes, where the normalized adjacency matrix is given by $\mathcal{S}(\tilde{\mathbf{A}})$. Equation (9) aligns with the propagation process of polynomial spectral-based GNNs presented in Equation (1): $\mathbf{Z} = \sum_{k=0}^{K} w_k \mathbf{P}^k \mathbf{X}$. Unlike the undirected methods, SDGAE maintains separate source and target embeddings, with corresponding polynomial coefficients $w_S^{(k)}$ and $w_T^{(k)}$. SDGAE is capable of approximating arbitrary graph filters in the spectral domain. Specifically, we denote the eigendecomposition of $\mathcal{S}(\tilde{\mathbf{A}})$ as $\mathcal{S}(\tilde{\mathbf{A}}) = \mathbf{U}_{\mathcal{S}}\mathbf{\Lambda}_{\mathcal{S}}\mathbf{U}_{\mathcal{S}}^\top$, where $\mathbf{U}_{\mathcal{S}}, \mathbf{\Lambda}_{\mathcal{S}} \in \mathbb{R}^{2n \times 2n}$ are the eigenvector and diagonal eigenvalue matrices, respectively. The spectral graph filter of SDGAE's encoder is then expressed as:

$$h(\mathcal{S}(\tilde{\mathbf{A}})) = \mathbf{U}_{\mathcal{S}}\left(\sum_{k=0}^{K} \mathbf{W}^{(k)}\mathbf{\Lambda}_{\mathcal{S}}^k\right)\mathbf{U}_{\mathcal{S}}^\top. \tag{10}$$

By learning the coefficients $w_S^{(k)}$ and $w_T^{(k)}$, SDGAE can approximate arbitrary graph filters $h(\mathcal{S}(\tilde{\mathbf{A}}))$ on directed graphs.

Implementation: In the implementation of SDGAE, we use two MLPs to initialize the source and target embeddings, i.e.,

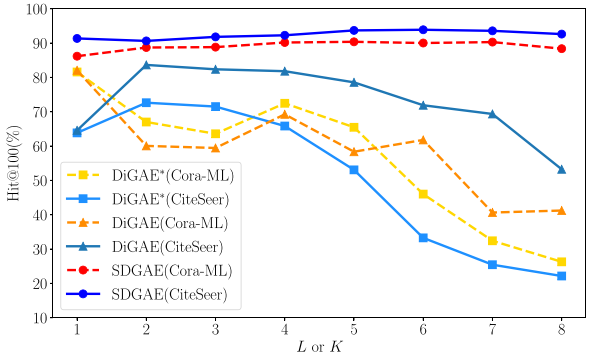


Fig. 4. Performance comparison of SDGAE, DiGAE, and DiGAE with residual connections (i.e., DiGAE*) on the Cora-ML and CiteSeer datasets, with varying numbers of convolutional layers or polynomial orders.

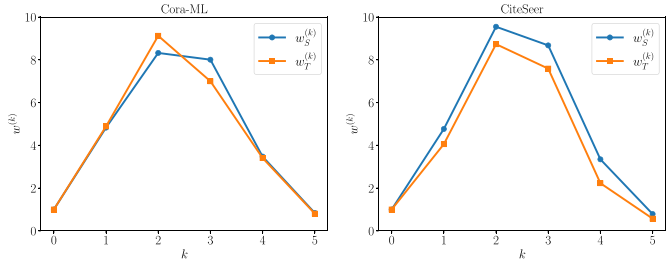


Fig. 5. Polynomial coefficients learned by SDGAE on the Cora-ML and CiteSeer datasets with $K = 5$.

$\mathbf{S}^{(0)} = \text{MLP}_S(\mathbf{X})$ and $\mathbf{T}^{(0)} = \text{MLP}_T(\mathbf{X})$, following the implementation of many spectral-based GNNs [37], [38]. For polynomial coefficients, we empirically find that directly learning $w_S^{(k)}$ and $w_T^{(k)}$ proves challenging, as different initialization strategies significantly impact the results. One possible explanation is that the coefficients must satisfy convergence constraints in polynomial approximations of filter functions [40]. To address this, we instead adopt an iterative formulation that implicitly learns the coefficients through the propagation process:

$$\mathbf{S}^{(k+1)} = \gamma_S^{(k)} \tilde{\mathbf{A}} \mathbf{T}^{(k)} + \mathbf{S}^{(k)}, \tag{11}$$

$$\mathbf{T}^{(k+1)} = \gamma_T^{(k)} \tilde{\mathbf{A}}^\top \mathbf{S}^{(k)} + \mathbf{T}^{(k)}. \tag{12}$$

where $\gamma_S^{(k)}$ and $\gamma_T^{(k)}$ are learnable scalar weights, initialized to one. These weights express the polynomial coefficients in Equation (9) through the iterative propagation process. For example, when the polynomial order is set to $K = 2$, the output can be expanded as:

$$\mathbf{S}^{(2)} = \mathbf{S}^{(0)} + \left(\gamma_S^{(1)} + \gamma_S^{(0)}\right) \tilde{\mathbf{A}} \mathbf{T}^{(0)} + \gamma_S^{(1)} \gamma_T^{(0)} \tilde{\mathbf{A}} \tilde{\mathbf{A}}^\top \mathbf{S}^{(0)}, \tag{13}$$

$$\mathbf{T}^{(2)} = \mathbf{T}^{(0)} + \left(\gamma_T^{(1)} + \gamma_T^{(0)}\right) \tilde{\mathbf{A}}^\top \mathbf{S}^{(0)} + \gamma_T^{(1)} \gamma_S^{(0)} \tilde{\mathbf{A}}^\top \tilde{\mathbf{A}} \mathbf{T}^{(0)}. \tag{14}$$

This expansion means the equivalent polynomial coefficients are: $w_S^{(0)} = 1, w_S^{(1)} = \gamma_S^{(1)} + \gamma_S^{(0)}, w_S^{(2)} = \gamma_S^{(1)} \gamma_T^{(0)}$ and $w_T^{(0)} = 1, w_T^{(1)} = \gamma_T^{(1)} + \gamma_T^{(0)}, w_T^{(2)} = \gamma_T^{(1)} \gamma_S^{(0)}$. Notably, the coefficients $w^{(k)}$ tend to decrease at higher orders. This is because the learned values of $\gamma^{(k)}$ generally lie within the range $(0, 1)$, and the product of multiple $\gamma^{(k)}$ terms at higher orders causes $w^{(k)}$ to diminish. The coefficients $w^{(k)}$ learned by SDGAE on real-world datasets, shown in Fig. 5, exhibit this trend. This convergence property contributes to the efficiency of learning polynomial filters.

For the experimental setting of SDGAE, we aligned its configuration with that of other baselines in DirLinkBench to ensure a fair comparison. Regarding the loss function and decoders, SDGAE uses BCE loss with three decoder options: $\sigma(\mathbf{s}_u \| \mathbf{t}_v)$, $\text{MLP}(\mathbf{s}_u \odot \mathbf{t}_v)$, and $\text{MLP}(\mathbf{s}_u \| \mathbf{t}_v)$. For the polynomial order K, we search over the set $\{3, 4, 5\}$. The MLP architecture, including the number of layers and hidden units, is consistent with the baseline methods. Detailed hyperparameter settings are provided in the Appendix, available online.

Time Complexity: The computational time complexity of SDGAE primarily stems from its propagation process, as defined in (9). Based on the iterative implementation shown in (11) and (12), the time complexity of SDGAE's propagation is $O(2Kmd)$, where m is the number of edges, K is the polynomial order, and d is the embedding dimension. This complexity scales linearly with the number of edges m. In contrast, DiGAE has a higher time complexity of $O(2Lmd + Lnd^2)$, where L is the number of layers and n is the number of nodes. This is due to the use of multiple learnable weight matrices, i.e., $\mathbf{W}_S^{(\ell)}$ and $\mathbf{W}_T^{(\ell)}$ as defined in (5) and (6). Thus, SDGAE achieves lower time complexity than DiGAE and shows faster training performance. Notably, SDGAE uses polynomial approximation and avoids explicit spectral decomposition. This makes it more suitable for sparse large graphs, while further improvements, such as sampled propagation [62] or decoupled implementations [40], may be useful for even larger graph settings.

Results: We report the directed link prediction results of SDGAE on DirLinkBench under the Hits@100 metric in Table VIII, and across seven different evaluation metrics in Table IX. Comprehensive results are also provided in the code repository. As shown in Table VIII, SDGAE achieves the best performance on 4 out of the 7 datasets under the Hits@100 metric and delivers competitive results on the remaining three. Moreover, as shown in Table IX, SDGAE ranks among the top two methods across all seven evaluation metrics. Notably, SDGAE significantly outperforms both DiGAE [14] and the spectral-based method GPRGNN [37], which is designed for undirected graphs. These results collectively demonstrate the substantial effectiveness of SDGAE for the task of directed link prediction.

VI. Analysis

In this section, we first analyze SDGAE with respect to the choice of polynomial order K and the learned coefficients, aiming to understand the reasons behind its improved performance compared to DiGAE. Meanwhile, we compare the performance of the representative graph Transformers on DirLinkBench. Next, we conduct a series of experimental investigations on various aspects of directed link prediction methods, including

TABLE X
PERFORMANCE OF DIGAE AND SDGAE UNDER DIFFERENT ADJACENCY MATRIX NORMALIZATIONS. BEST RESULTS ARE HIGHLIGHTED IN **BOLD**

Methods	Cora-ML	CiteSeer	Photo	Computers
$\text{DiGAE}_{(\alpha,\beta)}$	$82.06_{\pm 2.51}$	$83.64_{\pm 3.21}$	$55.05_{\pm 2.36}$	$41.55_{\pm 1.62}$
$\text{DiGAE}_{(-1/2,-1/2)}$	$70.89_{\pm 3.59}$	$71.60_{\pm 6.21}$	$38.75_{\pm 5.20}$	$32.73_{\pm 5.28}$
$\text{SDGAE}_{(-1/2,-1/2)}$	$\mathbf{90.37}_{\pm \mathbf{1.33}}$	$\mathbf{93.69}_{\pm \mathbf{3.68}}$	$\mathbf{68.84}_{\pm \mathbf{2.35}}$	$\mathbf{53.79}_{\pm \mathbf{1.56}}$

TABLE XI
COMPARATIVE PERFORMANCE WITH GRAPH TRANSFORMERS ON CORA-ML AND CITESEER. BEST RESULTS ARE HIGHLIGHTED IN **BOLD**

Methods	Cora-ML		CiteSeer	
	Hits@100	MRR	Hits@100	MRR
GCN	$70.15_{\pm 3.01}$	$14.25_{\pm 3.49}$	$80.36_{\pm 3.07}$	$18.28_{\pm 6.70}$
GAT	$79.72_{\pm 3.07}$	$9.30_{\pm 3.39}$	$85.88_{\pm 4.98}$	$16.71_{\pm 5.56}$
SAN	$65.12_{\pm 3.85}$	$10.15_{\pm 3.06}$	$73.62_{\pm 4.54}$	$19.25_{\pm 4.12}$
DiGT	$79.05_{\pm 3.92}$	$20.08_{\pm 4.69}$	$86.30_{\pm 4.17}$	$31.38_{\pm 3.39}$
SDGAE	$\mathbf{90.37}_{\pm \mathbf{1.33}}$	$\mathbf{28.45}_{\pm \mathbf{5.82}}$	$\mathbf{93.69}_{\pm \mathbf{3.68}}$	$\mathbf{42.50}_{\pm \mathbf{9.76}}$

feature inputs, loss functions and decoders, degree distribution, and negative sampling strategies, to offer new insights and highlight open challenges in this field.

A. Analysis of SDGAE

Compared with DiGAE: To investigate the performance differences between SDGAE and DiGAE in utilizing higher neighborhood information, Fig. 4 shows how SDGAE's performance changes with increasing polynomial order K, and how the performance of DiGAE and DiGAE* (DiGAE with residual connections) changes with increasing numbers of convolutional layers L. The results on both Cora-ML and CiteSeer datasets show that SDGAE's performance consistently improves with increasing K, achieving optimal results at $K = 5$. This trend aligns with the theoretical advantages of using polynomial filter approximations [37]. In contrast, the performance of DiGAE and DiGAE* declines as the number of layers increases. This is attributed to DiGAE's essentially GCN-like architecture, which suffers from well-known issues such as over-smoothing and difficulty training deeper networks [60]. Notably, although the iterative formulation of SDGAE in (11) and (12) may superficially resemble the addition of residual connections, it fundamentally differs by learning a polynomial filter as shown in (9). This fact is also supported by the experiments with DiGAE*, where simply adding residual connections fails to improve DiGAE's performance. Overall, SDGAE effectively uses higher neighborhood information by learning polynomial filter coefficients, enabling it to achieve superior performance as K increases.

We show in Fig. 5 the polynomial filter coefficients $w_S^{(k)}$ and $w_T^{(k)}$ learned by SDGAE on Cora-ML and CiteSeer. These coefficients are computed from the learned weights $\gamma_S^{(k)}$ and $\gamma_T^{(k)}$. We observe that the coefficients are largest at $k = 2$ and gradually decrease at higher orders (e.g., $k = 3, 4, 5$), which aligns with the expected behavior of polynomial expansions. SDGAE learns distinct sets of coefficients for different datasets, effectively adapting its polynomial filters to the underlying graph structure. This flexibility contrasts with DiGAE, which cannot learn specific filter functions.

Next, we analyze the impact of adjacency matrix normalization on DiGAE's performance. Table X presents the results of DiGAE and SDGAE under different normalization strategies. Here, $\text{DiGAE}_{(\alpha,\beta)}$ refers to the original settings used in the its paper [14], where the adjacency matrix is normalized as $\hat{\mathbf{D}}_{\text{out}}^{-\beta}\hat{\mathbf{A}}\hat{\mathbf{D}}_{\text{in}}^{-\alpha}$, with (α, β) searched over the grid $\{0.0, 0.2, 0.4, 0.6, 0.8\}^2$. Notably, this search space excludes the symmetric normalization $\hat{\mathbf{D}}_{\text{out}}^{-1/2}\hat{\mathbf{A}}\hat{\mathbf{D}}_{\text{in}}^{-1/2}$, which is used in both $\text{DiGAE}_{(-1/2,-1/2)}$ and $\text{SDGAE}_{(-1/2,-1/2)}$. The results show that applying symmetric normalization to DiGAE does not improve performance, suggesting that SDGAE's performance gains are not solely attributable to its normalization choice. Furthermore, as shown in Fig. 9(b), modifying DiGAE's decoder can improve its performance on certain datasets. However, DiGAE fails to achieve results comparable to SDGAE even with these modifications. These comparative experiments demonstrate that SDGAE significantly outperforms DiGAE, primarily due to its theoretically grounded design based on graph filters. By learning adaptive graph filters, SDGAE can better capture structural patterns across different datasets.

Compared with graph Transformers: Table XI reports the directed link prediction performance of representative graph Transformer models. SAN [63] is a representative spectral-based graph Transformer, while DiGT [32] is specifically designed for directed graphs. Detailed experimental setups are provided in the Appendix, available online. Overall, SDGAE consistently achieves substantially better results. In particular, SAN [63] relies on positional encodings derived from the symmetric Laplacian, which makes it less effective at capturing edge directionality. By contrast, DiGT [32] is tailored for directed graphs and can explicitly model directed information, leading to competitive performance among Transformer baselines. We also note that these graph Transformer backbones were not originally developed for the link prediction task and are constrained by the quadratic complexity of self-attention, which limits their scalability to large graphs. For example, on DirLinkBench, SAN and DiGT run out of GPU memory on the Photo and Computers datasets. Improving the scalability of graph Transformers and adapting them more effectively to directed link prediction, therefore, remains a promising direction for future work.

B. Feature Input

We examine the impact of different feature inputs on GNN performance in directed link prediction. Fig. 6 shows the results of various GNN methods using original features, in/out degrees, or random features as inputs across four datasets. Specifically, Fig. 6(a), 6(b), and 6(c) compare the performance of GNNs using original features versus in/out degrees on Cora-ML, Photo, and WikiCS, respectively. Fig. 6(d) presents a similar comparison using in/out degrees and random features on Slashdot, which lacks original node features. The results indicate that original features enhance GNN performance on certain datasets (e.g., Cora-ML

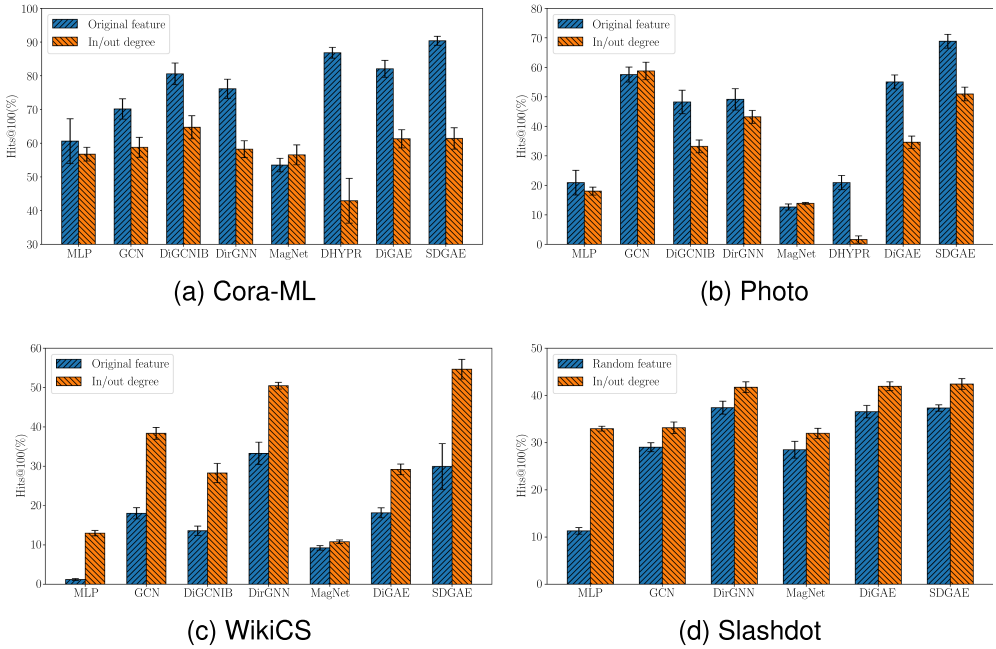


Fig. 6. Performance comparison of various GNN methods using original features, in/out degrees, or random features as input on four datasets.

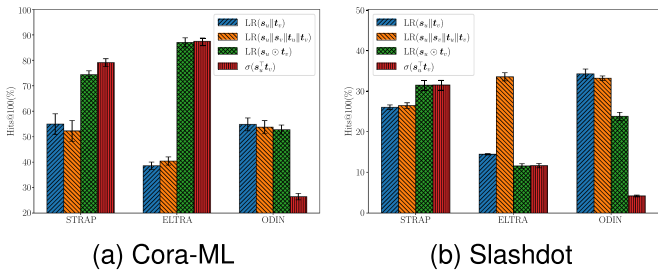


Fig. 7. Performance comparison of three embedding methods with four different decoders on the Cora-ML and Slashdot datasets.

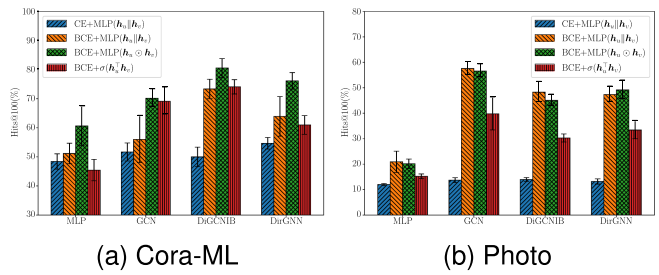


Fig. 8. Performance comparison of GNNs with various loss functions and decoders on Cora-ML and Photo datasets.

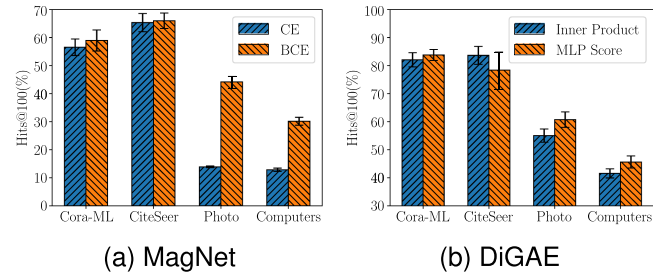


Fig. 9. Performance comparison of MagNet and DiGAE with different loss functions and decoders.

and Photo). However, for datasets like WikiCS, in/out degrees are more effective. For datasets without original features (e.g., Slashdot and Epinions), in/out degrees significantly outperform random features. These findings highlight the critical role of appropriate feature inputs in improving GNN performance on directed link prediction tasks. Enhancing feature quality remains an important direction for future research, particularly for datasets with weak or missing original features.

C. Loss Function and Decoder

We analyze the impact of different decoders and loss functions on the performance of directed link prediction methods. For the embedding methods STRAP, ELTRA, and ODIN, we compare their performance with four decoders: logistic regression with concatenation $\mathrm{LR}(\mathbf{s}_u \| \mathbf{t}_v)$, extended concatenation $\mathrm{LR}(\mathbf{s}_u \| \mathbf{s}_v \| \mathbf{t}_u \| \mathbf{t}_v)$, element-wise product $\mathrm{LR}(\mathbf{s}_u \odot \mathbf{t}_v)$, and inner product $\sigma(\mathbf{s}_u^\top \mathbf{t}_v)$ [9], [12], [13]. Results on the Cora-ML and Slashdot datasets are shown in Fig. 7. For single real-valued GNN methods, MLP, GCN, DiGCN-IB, and DirGNN, we evaluate different combinations of loss functions and decoders on the Cora-ML and Photo datasets, as shown in Fig. 8. The settings include CE loss with $\mathrm{MLP}(\mathbf{h}_u \| \mathbf{h}_v)$, and BCE loss with three decoders: $\mathrm{MLP}(\mathbf{h}_u \| \mathbf{h}_v)$, $\mathrm{MLP}(\mathbf{h}_u \odot \mathbf{h}_v)$, and inner product $\sigma(\mathbf{h}_u^\top \mathbf{h}_v)$. The results highlight the significant impact

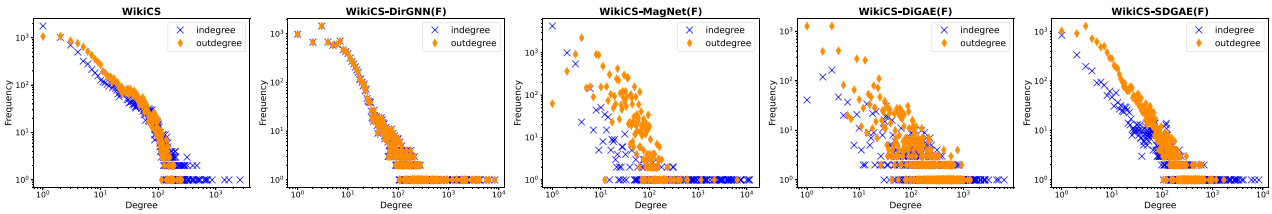


Fig. 10. Degree distribution of WikiCS graph and its reconstruction graph generated by four GNNs using the original node feature as feature inputs.

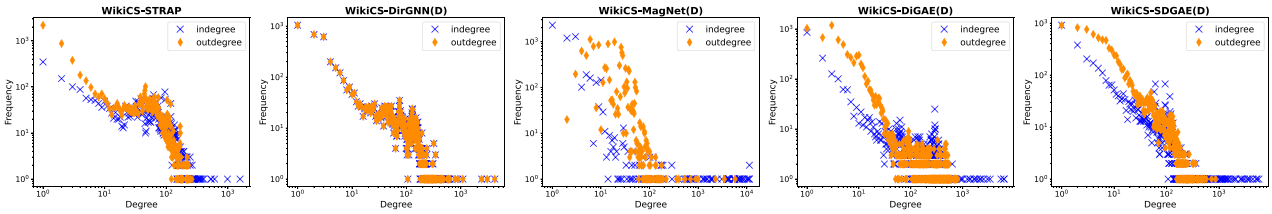


Fig. 11. Degree distribution of WikiCS's reconstruction graph generated by STRAP and four GNNs using the in/out degrees as feature inputs.

of both the decoder and loss function on model performance. For embedding methods, even with fixed embeddings, different decoders result in substantial performance variations. Notably, ELTRA and ODIN exhibit high sensitivity to decoder choices on the Slashdot dataset. For GNNs, the results show that BCE loss offers a consistent advantage over CE loss.

Additionally, we compare the performance of MagNet and DiGAE under different loss functions and decoders. In Fig. 9(a), we show that BCE loss consistently outperforms CE loss for MagNet, underscoring the limitations of prior approaches that rely on CE loss [24], [25], [26], [42]. Since link prediction is fundamentally a binary classification task, BCE loss is more appropriate, consistent with findings from undirected settings [45]. In Fig. 9(b), we compare two decoders for DiGAE, inner product and MLP score, and observe that the MLP-based decoder achieves superior performance across three datasets. These findings lead to two key insights: 1) Decoder design significantly affects model performance, and 2) BCE loss is better suited for link prediction tasks than CE loss. Furthermore, the poor performance of complex-valued methods (e.g., MagNet and DUPLEX) may be partly attributed to their reliance on CE loss and suboptimal decoder choices.

D. Degree Distribution

We evaluate how well different models preserve the asymmetry of directed graphs by analyzing their degree distributions. Following STRAP [9], we compute the predicted probability for every edge and select the top m' edges, where m' is the number of edges in the training graph, to reconstruct the graph. Fig. 10 compares the true in-/out-degree distributions of the WikiCS training graph with those of reconstructed graphs generated by four GNNs (DirGNN [21], MagNet [24], DiGAE [14], and SDGAE), using original node features as input. Fig. 11 presents a similar comparison, but the reconstructed graphs are generated by STRAP and the same four GNNs using in/out degrees as feature inputs. The results show that STRAP and SDGAE most accurately preserve the degree distributions, with STRAP performing exceptionally well in capturing in-degree components, explaining its strong performance on WikiCS. And DirGNN, using the decoder $\text{MLP}\big(\mathbf{h}_u \odot \mathbf{h}_v\big)$, produces identical in-/out-degree distributions but still captures in-degree components correctly. In contrast, MagNet fails to learn meaningful degree distributions, resulting in poor performance. Moreover, when GNNs are provided with in/out degrees as input features, they better preserve the degree distribution, aligning with their improved WikiCS performance (see code repository for detailed results). Finally, a direct comparison between DiGAE and SDGAE reveals that SDGAE more effectively maintains the degree distribution, further demonstrating its advantage. These findings reinforce the importance of preserving asymmetry in directed link prediction, as discussed in Section III-A. They also highlight an underexplored challenge: the need for GNNs to better preserve in-/out-degree distributions, a task that embedding methods like STRAP handle more effectively.

E. Negative Sampling Strategy

We present a performance comparison of various GNNs trained using different negative sampling strategies on the Cora-ML and CiteSeer datasets in Table XII. Results are reported using the Hits@100 metric, with the best results highlighted in bold. In this context, "each run" strategy refers to the default setting in DirLinkBench, where a random negative sample is generated for each run and shared across all models. In contrast, "each epoch" represents a strategy where different models randomly sample negative edges in each training epoch. In both settings, the positive sample splits and test sets remain consistent for fair comparison. The results demonstrate that the choice of negative sampling strategy during training can significantly

TABLE XII
PERFORMANCE COMPARISON OF VARIOUS GNNS USING DIFFERENT NEGATIVE SAMPLING STRATEGIES ON THE CORA-ML AND CITESEER DATASETS. RESULTS ARE REPORTED UNDER THE **HITS@100** METRIC, WITH THE BEST RESULTS HIGHLIGHTED IN **BOLD**

Dataset	Sample	MLP	GCN	DiGCNIB	DirGNN	MagNet	DiGAE	SDGAE
Cora-ML	each run	**60.61±6.64**	**70.15±3.01**	**80.57±3.21**	**76.13±2.85**	**56.54±2.95**	**82.06±2.51**	**90.37±1.33**
	each epoch	34.15±3.54	59.86±9.89	56.74±4.08	49.89±3.59	54.79±2.98	79.76±3.28	89.71±2.36
CiteSeer	each run	**70.27±3.40**	**80.36±3.07**	**85.32±3.70**	**76.83±4.24**	65.32±3.26	83.64±3.21	**93.69±3.68**
	each epoch	66.92±6.50	69.48±6.60	61.69±6.87	53.8±12.41	**70.56±2.06**	**87.32±3.79**	92.12±3.96

affect model performance, particularly for single real-valued GNNs, where performance declines noticeably under the "each epoch" strategy. In contrast, the "each run" strategy tends to improve performance across most GNN models. These findings underscore the importance of further research into negative sampling techniques. For example, heuristic-based approaches have been proposed for undirected graphs as alternatives to random sampling [45], and similar methods could be explored for directed graphs.

VII. CONCLUSION AND FUTURE DIRECTIONS

This paper presents a unified framework for evaluating the expressiveness of directed link prediction methods, highlighting the theoretical importance of dual embeddings and decoder design. To address the lack of standardized benchmarks, we introduce DirLinkBench, a robust benchmark with diverse real-world directed graphs, standardized data splits, varied feature initialization strategies, and comprehensive evaluation metrics. Using DirLinkBench, we show that current methods often perform inconsistently across datasets, and that seemingly simple design choices, such as feature inputs, loss functions, decoders, and negative sampling, can substantially affect performance. We then revisit the DiGAE model and show that its graph convolution is theoretically equivalent to GCN on an undirected bipartite graph. Building on this insight, we propose SDGAE, a Spectral Directed Graph Auto-Encoder that uses polynomial approximation to learn graph filters for directed graphs. SDGAE achieves SOTA average performance and better preserves directed structural properties.

Our findings also suggest several important directions for future research. First, although this work rethinks directed link prediction mainly through decoder architectures already used in prior methods, designing more expressive and efficient decoders remains an important open problem, especially for asymmetric and complex-valued models. Second, directed GNN architectures still need to better capture and preserve graph asymmetry, such as in- and out-degree distribution. Beyond the scope of this paper, several emerging directions may further broaden the study of directed link prediction. Recent progress in graph prompting and instruction-guided graph learning has shown promise for multi-task adaptation and flexible task steering [64], [65], [66], [67]. Meanwhile, dynamic, sociotechnical, and hypergraph-based graph learning settings are becoming increasingly important in real-world applications [68], [69], [70]. Exploring how these paradigms can be combined with directed link prediction tasks and asymmetry-aware graph models is a promising direction for future work.

ACKNOWLEDGMENTS

The authors would like to acknowledge that this work was partially conducted at Gaoling School of Artificial Intelligence, Beijing Key Laboratory of Research on Large Models and Intelligent Governance, Engineering Research Center of Next-Generation Intelligent Search and Recommendation, MOE, and Pazhou Laboratory (Huangpu), Guangzhou, Guangdong 510555, China. The authors would also like to acknowledge that part of this work was also completed while Mingguo He was a visiting student at the Technical University of Munich and later a Research Fellow with the National University of Singapore.

REFERENCES

[1] J. Leskovec and R. Sosič, "Snap: A general-purpose network analysis and graph-mining library," *ACM Trans. Intell. Syst. Technol.*, vol. 8, no. 1, pp. 1–20, 2016.
[2] L. Page, S. Brin, R. Motwani, and T. Winograd, "The PageRank citation ranking: Bringing order to the web," Stanford InfoLab, Stanford Univ., Tech. Rep. 1999-66, 1999.
[3] D. Liben-Nowell and J. Kleinberg, "The link prediction problem for social networks," in *Proc. 12th Int. Conf. Inf. Knowl. Manage.*, 2003, pp. 556–559.
[4] S. Rendle, C. Freudenthaler, Z. Gantner, and L. Schmidt-Thieme, "BPR: Bayesian personalized ranking from implicit feedback," in *Proc. 25th Conf. Uncertainty Artif. Intell.*, 2009, pp. 452–461.
[5] M. H. Bhuyan, D. K. Bhattacharyya, and J. K. Kalita, "Network anomaly detection: Methods, systems and tools," *IEEE Commun. Surveys Tuts.*, vol. 16, no. 1, pp. 303–336, First Quarter 2014.
[6] M. Ou, P. Cui, J. Pei, Z. Zhang, and W. Zhu, "Asymmetric transitivity preserving graph embedding," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2016, pp. 1105–1114.
[7] C. Zhou, Y. Liu, X. Liu, Z. Liu, and J. Gao, "Scalable graph embedding for asymmetric proximity," in *Proc. AAAI Conf. Artif. Intell.*, 2017, vol. 31, no. 1, pp. 2942–2948.
[8] Z. Zhang, P. Cui, X. Wang, J. Pei, X. Yao, and W. Zhu, "Arbitrary-order proximity preserved network embedding," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2018, pp. 2778–2786.
[9] Y. Yin and Z. Wei, "Scalable graph embeddings via sparse transpose proximities," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2019, pp. 1429–1437.
[10] M. Khosla, J. Leonhardt, W. Nejdl, and A. Anand, "Node representation learning for directed graphs," in *Proc. Joint Eur. Conf. Mach. Learn. Knowl. Discov. Databases*, 2020, pp. 395–411.
[11] S. Zhu, J. Li, H. Peng, S. Wang, and L. He, "Adversarial directed graph embedding," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 4741–4748.
[12] M. R. Hamedani, J.-S. Ryu, and S.-W. Kim, "ELTRA: An embedding method based on learning-to-rank to preserve asymmetric information in directed graphs," in *Proc. 32nd ACM Int. Conf. Inf. Knowl. Manage.*, 2023, pp. 2116–2125.
[13] H. Yoo, Y.-C. Lee, K. Shin, and S.-W. Kim, "Disentangling degree-related biases and interest for out-of-distribution generalized directed network embedding," in *Proc. ACM Web Conf. 2023*, 2023, pp. 231–239.

[14] G. Kollias, V. Kalantzis, T. Idé, A. Lozano, and N. Abe, "Directed graph auto-encoders," in *Proc. AAAI Conf. Artif. Intell.*, 2022, pp. 7211–7219.
[15] L. Liu, K.-J. Chen, and Z. Liu, "Collaborative bi-aggregation for directed graph embedding," *Neural Netw.*, vol. 164, pp. 707–718, 2023.
[16] S. Virinchi and A. Saladi, "Blade: Biased neighborhood sampling based graph neural network for directed graphs," in *Proc. 16th ACM Int. Conf. Web Search Data Mining*, 2023, pp. 42–50.
[17] G. Salha, S. Limnios, R. Hennequin, V.-A. Tran, and M. Vazirgiannis, "Gravity-inspired graph autoencoders for directed link prediction," in *Proc. 28th ACM Int. Conf. Inf. Knowl. Manage.*, 2019, pp. 589–598.
[18] H. Zhou, A. Chegu, S. S. Sohn, Z. Fu, G. De Melo, and M. Kapadia, "D-HYPR: Harnessing neighborhood modeling and asymmetry preservation for digraph representation learning," in *Proc. 31st ACM Int. Conf. Inf. Knowl. Manage.*, 2022, pp. 2732–2742.
[19] Z. Tong, Y. Liang, C. Sun, D. S. Rosenblum, and A. Lim, "Directed graph convolutional network," 2020, *arXiv:2004.13970*.
[20] Z. Tong, Y. Liang, C. Sun, X. Li, D. Rosenblum, and A. Lim, "Digraph inception convolutional networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, vol. 33, pp. 17907–17918.
[21] E. Rossi, B. Charpentier, F. Di Giovanni, F. Frasca, S. Günnemann, and M. M. Bronstein, "Edge directionality improves learning on heterophilic graphs," in *Proc. Learn. Graphs Conf.*, 2024, pp. 25-1–25-27.
[22] C. Koke and D. Cremers, "HoloNets: Spectral convolutions do extend to directed graphs," in *Proc. Int. Conf. Learn. Representations*, 2024.
[23] J. Huang et al., "Exploring the role of node diversity in directed graph representation learning," in *Proc. 33rd Int. Joint Conf. Artif. Intell.*, 2024, pp. 2072–2080.
[24] X. Zhang, Y. He, N. Brugnone, M. Perlmutter, and M. Hirn, "Magnet: A neural network for directed graphs," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, vol. 34, pp. 27003–27015.
[25] X. Li et al., "LightDiC: A simple yet effective approach for large-scale digraph representation learning," *Proc. VLDB Endowment*, vol. 17, no. 7, pp. 1542–1551, 2024.
[26] Z. Ke, H. Yu, J. Li, and H. Zhang, "DUPLEX: Dual GAT for complex embedding of directed graphs," in *Proc. Int. Conf. Mach. Learn.*, 2024, pp. 23430–23448.
[27] J. Tang, M. Qu, M. Wang, M. Zhang, J. Yan, and Q. Mei, "LINE: Large-scale information network embedding," in *Proc. 24th Int. Conf. World Wide Web*, 2015, pp. 1067–1077.
[28] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learn. Representations*, 2017.
[29] L. Katz, "A new status index derived from sociometric analysis," *Psychometrika*, vol. 18, no. 1, pp. 39–43, 1953.
[30] G. H. Golub and C. F. van Loan, *Matrix Computations*, 4th ed.Baltimore, MD, USA: Johns Hopkins Univ. Press, 2013.
[31] F. Monti, K. Otness, and M. M. Bronstein, "MotifNet: A motif-based graph convolutional network for directed graphs," in *Proc. IEEE Data Sci. Workshop*, 2018, pp. 225–228.
[32] Q. Wang, G. Kollias, V. Kalantzis, N. Abe, and M. J. Zaki, "Directed graph transformers," *Trans. Mach. Learn. Res.*, 2024. [Online]. Available: https://openreview.net/forum?id=otTFPjziiK
[33] S. Geisler, Y. Li, D. J. Mankowitz, A. T. Cemgil, S. Günnemann, and C. Paduraru, "Transformers meet directed graphs," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 11144–11172.
[34] S. Maskey, R. Paolino, A. Bacho, and G. Kutyniok, "A fractional graph Laplacian approach to oversmoothing," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, vol. 36, pp. 13022–13063.
[35] D. Bo, X. Wang, Y. Liu, Y. Fang, Y. Li, and C. Shi, "A survey on spectral graph neural networks," 2023, *arXiv:2302.05631*.
[36] M. Defferrard, X. Bresson, and P. Vandergheynst, "Convolutional neural networks on graphs with fast localized spectral filtering," in *Proc. Adv. Neural Inf. Process. Syst.*, 2016, vol. 29, pp. 3837–3845.
[37] E. Chien, J. Peng, P. Li, and O. Milenkovic, "Adaptive universal generalized PageRank graph neural network," in *Proc. Int. Conf. Learn. Representations*, 2021.
[38] M. He, Z. Wei, Z. Huang, and H. Xu, "BernNet: Learning arbitrary graph spectral filters via Bernstein approximation," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, vol. 34, pp. 14239–14251.
[39] X. Wang and M. Zhang, "How powerful are spectral graph neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 23341–23362.
[40] M. He, Z. Wei, and J.-R. Wen, "Convolutional neural networks on graphs with Chebyshev approximation, revisited," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, vol. 35, pp. 7264–7276.
[41] M. Zhang and Y. Chen, "Link prediction based on graph neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, vol. 31, pp. 5165–5175.
[42] Y. He, X. Zhang, J. Huang, B. Rozemberczki, M. Cucuringu, and G. Reinert, "PyTorch geometric signed directed: A software package on graph neural networks for signed and directed graphs," in *Proc. Learn. Graphs Conf.*, 2023, pp. 1–12.
[43] S. Fiorini, S. Coniglio, M. Ciavotta, and E. Messina, "SigmaNet: One Laplacian to rule them all," in *Proc. AAAI Conf. Artif. Intell.*, 2023, pp. 7568–7576.
[44] L. Lin and J. Gao, "A magnetic framelet-based convolutional neural network for directed graphs," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process.*, 2023, pp. 1–5.
[45] J. Li et al., "Evaluating graph neural networks for link prediction: Current pitfalls and new benchmarking," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, vol. 36, pp. 3853–3866.
[46] M. Fey and J. E. Lenssen, "Fast graph representation learning with PyTorch geometric," in *Proc. Int. Conf. Learn. Representations Workshop Representation Learn. Graphs Manifolds*, 2019.
[47] A. K. McCallum, K. Nigam, J. Rennie, and K. Seymore, "Automating the construction of internet portals with machine learning," *Inf. Retrieval*, vol. 3, pp. 127–163, 2000.
[48] A. Bojchevski and S. Günnemann, "Deep Gaussian embedding of graphs: Unsupervised inductive learning via ranking," in *Proc. Int. Conf. Learn. Representations*, 2018.
[49] P. Sen, G. Namata, M. Bilgic, L. Getoor, B. Galligher, and T. Eliassi-Rad, "Collective classification in network data," *AI Mag.*, vol. 29, no. 3, pp. 93–93, 2008.
[50] O. Shchur, M. Mumme, A. Bojchevski, and S. Günnemann, "Pitfalls of graph neural network evaluation," in *Proc. Relational Representation Learn. Workshop, NeurIPS 2018*, 2018.
[51] P. Mernyei and C. Cangea, "Wiki-CS: A Wikipedia-based benchmark for graph neural networks," 2020, *arXiv:2007.02901*.
[52] B. Ordozgoiti, A. Matakos, and A. Gionis, "Finding large balanced subgraphs in signed networks," in *Proc. Web Conf.*, 2020, pp. 1378–1388.
[53] P. Massa and P. Avesani, "Controversial users demand local trust metrics: An experimental study on epinions. com community," in *Proc. AAAI Conf. Artif. Intell.*, 2005, vol. 1, pp. 121–126.
[54] J. Gasteiger, A. Bojchevski, and S. Günnemann, "Predict then propagate: Graph neural networks meet personalized pagerank," in *Proc. Int. Conf. Learn. Representations*, 2019.
[55] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. Int. Conf. Learn. Representations*, 2018.
[56] W. Hu et al., "Open graph benchmark: Datasets for machine learning on graphs," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, vol. 33, pp. 22118–22133.
[57] Y. Yang, R. N. Lichtenwalter, and N. V. Chawla, "Evaluating link prediction methods," *Knowl. Inf. Syst.*, vol. 45, pp. 751–782, 2015.
[58] B. Weisfeiler and A. A. Leman, "The reduction of a graph to canonical form and the algebra which appears therein," *Nauchno-Technicheskaya Informatsia*, vol. 9, pp. 12–16, 1968.
[59] J. Bang-Jensen and G. Z. Gutin, *Digraphs: Theory, Algorithms and Applications*. Berlin, Germany: Springer, 2008.
[60] J. Peng, R. Lei, and Z. Wei, "Beyond over-smoothing: Uncovering the trainability challenges in deep graph neural networks," in *Proc. 33rd ACM Int. Conf. Inf. Knowl. Manage.*, 2024, pp. 1878–1887.
[61] M. Zhu, X. Wang, C. Shi, H. Ji, and P. Cui, "Interpreting and unifying graph neural networks with an optimization framework," in *Proc. Web Conf. 2021*, 2021, pp. 1215–1226.
[62] H. Ding, Z. Wei, and Y. Ye, "Large-scale spectral graph neural networks via Laplacian sparsification," in *Proc. 31st ACM SIGKDD Conf. Knowl. Discov. Data Mining V. 1*, 2025, pp. 213–223.
[63] D. Kreuzer, D. Beaini, W. Hamilton, V. Létourneau, and P. Tossou, "Rethinking graph transformers with spectral attention," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, vol. 34, pp. 21618–21629.
[64] X. Sun, H. Cheng, J. Li, B. Liu, and J. Guan, "All in one: Multi-task prompting for graph neural networks," in *Proc. 29th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2023, pp. 2120–2131.
[65] X. Sun, J. Zhang, X. Wu, H. Cheng, Y. Xiong, and J. Li, "Graph prompt learning: A comprehensive survey and beyond," 2023, *arXiv:2311.16534*.
[66] Q. Wang, X. Sun, and H. Cheng, "Does graph prompt work? A data operation perspective with theoretical analysis," in *Proc. Int. Conf. Mach. Learn.*, 2025, pp. 64377–64402.
[67] J. Li, X. Sun, Y. Li, Z. Li, H. Cheng, and J. X. Yu, "Graph intelligence with large language models and prompt learning," in *Proc. 30th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2024, pp. 6545–6554.

[68] X. Sun et al., "Structure learning via meta-hyperedge for dynamic rumor detection," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 9, pp. 9128–9139, Sep. 2023.
[69] X. Sun et al., "Self-supervised hypergraph representation learning for sociological analysis," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 11, pp. 11860–11871, Nov. 2023.
[70] Z. Shu, X. Sun, and H. Cheng, "When LLM meets hypergraph: A sociological analysis on personality via online social networks," in *Proc. 33rd ACM Int. Conf. Inf. Knowl. Manage.*, 2024, pp. 2087–2096.

Mingguo He (Member, IEEE) received the BE degree in software engineering from Central South University, Changsha, China, in 2020, and the PhD degree in artificial intelligence from the Gaoling School of Artificial Intelligence, Renmin University of China, Beijing, China, in 2025. He is currently a research fellow with the School of Computing, National University of Singapore, Singapore. His research interests include spectral graph neural networks, graph transformers, graph learning, and AI for Science.

Yuhe Guo received the BS and MS degrees from the Renmin University of China. She is currently working toward the PhD degree with the Gaoling School of Artificial Intelligence, Renmin University of China, Beijing, China. Her research interests include graph neural networks, particularly spectral GNNs, equivariant and invariant GNNs, graph rewiring, expressive GNNs, and the theoretical foundations of graph learning.

Yanping Zheng received the PhD degree in artificial intelligence from the Gaoling School of Artificial Intelligence, Renmin University of China, Beijing, China, in 2024. She is currently a postdoctoral researcher with the Gaoling School of Artificial Intelligence, Renmin University of China. Her research interests include graph learning algorithms, efficient algorithms for graph neural networks, and dynamic graph representation learning.

Zhewei Wei (Member, IEEE) received the BSc degree from the School of Mathematical Sciences, Peking University, in 2008, and the PhD degree from the Department of Computer Science and Engineering, Hong Kong University of Science and Technology, in 2012. He is currently a professor with the Gaoling School of Artificial Intelligence, Renmin University of China. His research interests include graph algorithms, massive data algorithms, and streaming algorithms. He was the proceeding chair of SIGMOD/PODS2020 and ICDT2021.

Stephan Günnemann received the PhD degree in computer science from RWTH Aachen University, Aachen, Germany, in 2012. He was with Carnegie Mellon University, first as a postdoctoral fellow and then a senior researcher. He researched for the Simon Fraser University in Canada, and for the Research & Technology Center of the Siemens AG. In 2015, he founded a DFG Emmy-Noether Research Group, Computer Science Department, Technical University of Munich (TUM). In 2016, he became professor with TUM where he is leading the Data Analytics and Machine Learning Group. In 2020, he was appointed as the director with the Munich Data Science Institute. His research focuses on robust machine learning approaches, ensuring a reliable use of AI techniques in science and industry.

Xiaokui Xiao (Fellow, IEEE) received the PhD degree in computer science from the Chinese University of Hong Kong. He is currently a professor with the School of Computing, National University of Singapore (NUS), Singapore. He was an associate professor with Nanyang Technological University, Singapore. His research interests include data management, especially on data privacy and algorithms for large data. He was the recipient of the Best Research Paper Award in VLDB 2021, and the ACM SIGMOD Research Highlight Award in 2022. He is also an ACM distinguished member.